

```
using System;
using System.Data;
using System.controller.data;
using System.Data.SqlClient;
using controller.system;
using controller.system.I;
using controller.system.DBFactory;
namespace controller.system
{
    public class assetmanageBLL
    {
        private static readonly Iassetmanage _ = Factory.assetmanageInstance();
        public assetmanageBLL()
        {
        }
        public static int Insert(assetmanage assetmanage)
        {
            if (assetmanage == null)
                return 0;
            return assetmanageBLL._.Insert(assetmanage);
        }
        public static int Insert(SqlTransaction sp, assetmanage assetmanage)
        {
            if (assetmanage == null)
                return 0;
            return assetmanageBLL._.Insert(sp, assetmanage);
        }
        public static int Update(assetmanage assetmanage)
        {
            if (assetmanage == null)
                return 0;
            return assetmanageBLL._.Update(assetmanage);
        }
        public static int Update(SqlTransaction sp, assetmanage assetmanage)
        {
            if (assetmanage == null)
                return 0;
            return assetmanageBLL._.Update(sp, assetmanage);
        }
        public static int Delete(int am_id)
        {
            if (am_id < 0)
                return 0;
            return assetmanageBLL._.Delete(am_id);
        }
        public static int Delete(assetmanage assetmanage)
        {
            return assetmanageBLL._.Delete(assetmanage);
        }
        public static int Delete(SqlTransaction sp, int am_id)
```

```
{
    if (am_id < 0)
        return 0;
    return assetmanageBLL. Delete(sp, am_id);
}

public int Delete(SqlTransaction sp, assetmanage assetmanage)
{
    return assetmanageBLL. Delete(sp, assetmanage);
}

public static assetmanage Select(DataRow row)
{
    return assetmanageBLL. Select(row);
}

public static assetmanage Select(IDataReader dr)
{
    return assetmanageBLL. Select(dr);
}

public static assetmanage Select(int am_id)
{
    if (am_id < 0)
        return null;
    return assetmanageBLL. Select(am_id);
}

public static assetmanage Select(int am_id, bool bParentTable, bool
bChildrenTable)
{
    return assetmanageBLL. Select(am_id, bParentTable, bChildrenTable);
}

public disease Select(int em_id)
{
    return this.Select(em_id, false, false);
}

public disease Select(int em_id, bool bParentTable, bool bChildrenTable)
{
    .disease obj = null;
    Sqlconcontrol[] param = {
new Sqlconcontrol("@em_id", SqlDbType.Int)
};

    param[0].Value = em_id;
    string sqlCommand = "control.diseaseSelect";
    using (SqlDataReader dr = SqlHelper.ExecuteReader(Conn.SqlConn,
CommandType.StoredProcedure, sqlCommand, param))
    {
        while (dr.Read())
        {
            obj = this.Select(dr);
        }
    }
    this.Select(obj, bParentTable, bChildrenTable);
    return obj;
}
```

```

    }
    public void Select(control.disease obj, bool bParentTable, bool
bChildrenTable)
    {
        if (bParentTable)
        {
            obj.nation = nationBLL.Select(obj.em_na_id);
            obj.visage = visageBLL.Select(obj.em_v_id);
            obj.province = provinceBLL.Select(obj.em_p_id);
            obj.native = nativeBLL.Select(obj.em_n_id);
            obj.learn = learnBLL.Select(obj.em_l_id);
            obj.department = departmentBLL.Select(obj.em_dept_id);
        }
        if (bChildrenTable)
        {
            obj.users = usersBLL.Select("u_em_id=" + obj.em_id.ToString());
            foreach (users item in obj.users)
            {
                item.control.disease = obj;
            }
        }
    }
    public void nation(control.disease.disease)
    {
        .disease.nation = nationBLL.Select(control.disease.em_na_id);
    }
    public void visage(control.disease.disease)
    {
        .disease.visage = visageBLL.Select(control.disease.em_v_id);
    }
    public void province(control.disease.disease)
    {
        .disease.province = provinceBLL.Select(control.disease.em_p_id);
    }
    public void native(control.disease.disease)
    {
        .disease.native = nativeBLL.Select(control.disease.em_n_id);
    }
    public void learn(control.disease.disease)
    {
        .disease.learn = learnBLL.Select(control.disease.em_l_id);
    }
    public void department(control.disease.disease)
    {
        .disease.department
departmentBLL.Select(control.disease.em_dept_id);
    }
    public void users(control.disease.disease)
    {
        .disease.users
=
usersBLL.Select("u_em_id="

```

```
+.disease.em_id.ToString());
    }
    public IList<control.disease> Select()
    {
        return this.Select(false, false);
    }
    public IList<control.disease> Select(bool bParentTable, bool
bChildrenTable)
    {
        IList<control.disease> list = new List<control.disease>();
        string sqlCommand = "control.diseaseSelectAll";
        using (SqlDataReader dr = SqlHelper.ExecuteReader(Conn.SqlConn,
CommandType.StoredProcedure, sqlCommand))
        {
            while (dr.Read())
            {
                list.Add(this.Select(dr));
            }
        }
        foreach (control.disease obj in list)
        {
            this.Select(obj, bParentTable, bChildrenTable);
        }
        return list;
    }
    public IList<control.disease> Select(string where)
    {
        return this.Select(where, false, false);
    }
    public IList<control.disease> Select(string where, bool bParentTable, bool
bChildrenTable)
    {
        IList<control.disease> list = new List<control.disease>();
        Sqlconcontrol[] param = {
new Sqlconcontrol("@where", SqlDbType.VarChar, 8000)
};
        param[0].Value = where;
        string sqlCommand = "control.diseaseSelectByParams";
        using (SqlDataReader dr = SqlHelper.ExecuteReader(Conn.SqlConn,
CommandType.StoredProcedure, sqlCommand, param))
        {
            while (dr.Read())
            {
                list.Add(this.Select(dr));
            }
        }
        foreach (control.disease obj in list)
        {
            this.Select(obj, bParentTable, bChildrenTable);
        }
    }
}
```

```

        return list;
    }

    public IList<control.disease> Select(string foreignFieldName, int
foreignFieldValue)
    {
        return this.Select(foreignFieldName, foreignFieldValue, false,
false);
    }

    public IList<control.disease> Select(string foreignFieldName, int
foreignFieldValue, bool bParentTable, bool bChildrenTable)
    {
        return this.Select(string.Format("{0}='{1}'", foreignFieldName,
foreignFieldValue), bParentTable, bChildrenTable);
    }

    public void Select(string where, out int recordCount)
    {
        string sqlCommand = "control.diseaseCountByWhere";
        Sqlconcontrol[] param = {
new Sqlconcontrol("@where", SqlDbType.VarChar, 8000),
new Sqlconcontrol("@recordCount", SqlDbType.Int)
};

        param[0].Value = where;
        param[1].Direction = concontrolDirection.Output;
        SqlHelper.ExecuteNonQuery(Conn.SqlConn, CommandType.StoredProcedure,
sqlCommand, param);
        recordCount = Convert.ToInt32(param[1].Value);
    }

    public IList<control.disease> Select(int pageSize, int pageIndex, string
where)
    {
        return this.Select(pageSize, pageIndex, where, false, false);
    }

    public IList<control.disease> Select(int pageSize, int pageIndex, string
where, bool bParentTable, bool bChildrenTable)
    {
        IList<control.disease> list = new List<control.disease>();
        string sqlCommand = "control.diseaseSelectByPagerParams";
        Sqlconcontrol[] param = {
new Sqlconcontrol("@pageSize", SqlDbType.Int),
new Sqlconcontrol("@pageIndex", SqlDbType.Int),
new Sqlconcontrol("@where", SqlDbType.VarChar, 8000)
};

        param[0].Value = pageSize;
        param[1].Value = pageIndex;
        param[2].Value = where;
        using (SqlDataReader dr = SqlHelper.ExecuteReader(Conn.SqlConn,
CommandType.StoredProcedure, sqlCommand, param))
        {
            while (dr.Read())
            {

```

```
        list.Add(this.Select(dr));
    }
}
foreach (control.disease obj in list)
{
    this.Select(obj, bParentTable, bChildrenTable);
}
return list;
}

public IList<control.disease> Select(CommandType commandType, string
sqlCommand, params Sqlconcontrol[] param)
{
    return this.Select(false, false, commandType, sqlCommand, param);
}

public IList<control.disease> Select(bool bParentTable, bool
bChildrenTable, CommandType commandType, string sqlCommand, params Sqlconcontrol[]
param)
{
    IList<control.disease> list = new List<control.disease>();
    using (SqlDataReader dr = SqlHelper.ExecuteReader(Conn.SqlConn,
commandType, sqlCommand, param))
    {
        while (dr.Read())
        {
            list.Add(this.Select(dr));
        }
    }
    foreach (control.disease obj in list)
    {
        this.Select(obj, bParentTable, bChildrenTable);
    }
    return list;
}

public bool Exists(int em_id)
{
    Sqlconcontrol[] param = {
        new Sqlconcontrol("@em_id", SqlDbType.Int)
    };
    param[0].Value = em_id;
    string sqlCommand = "control.diseaseIsExist";
    int a = Convert.ToInt32(SqlHelper.ExecuteScalar(Conn.SqlConn,
CommandType.StoredProcedure, sqlCommand, param));
    if (a > 0)
    {
        return true;
    }
    else
    {
        return false;
    }
}
```

```
    }
}

public static void Select(assetmanage obj, bool bParentTable, bool
bChildrenTable)
{
    assetmanageBLL._.Select(obj, bParentTable, bChildrenTable);
}

public static void department(assetmanage assetmanage)
{
    assetmanageBLL._.department(assetmanage);
}

public static IList<assetmanage> Select()
{
    return assetmanageBLL._.Select();
}

public static IList<assetmanage> Select(bool bParentTable, bool
bChildrenTable)
{
    return assetmanageBLL._.Select(bParentTable, bChildrenTable);
}

public static IList<assetmanage> Select(string where)
{
    return assetmanageBLL._.Select(where);
}

public static IList<assetmanage> Select(string where, bool bParentTable,
bool bChildrenTable)
{
    return assetmanageBLL._.Select(where, bParentTable, bChildrenTable);
}

public static IList<assetmanage> Select(string foreignFieldName, int
foreignFieldValue)
{
    return assetmanageBLL._.Select(foreignFieldName, foreignFieldValue);
}

public static IList<assetmanage> Select(string foreignFieldName, int
foreignFieldValue, bool bParentTable, bool bChildrenTable)
{
    return assetmanageBLL._.Select(foreignFieldName, foreignFieldValue,
bParentTable, bChildrenTable);
}

public static void Select(string where, out int recordCount)
{
    assetmanageBLL._.Select(where, out recordCount);
}

public static IList<assetmanage> Select(int pageSize, int pageIndex,
string where)
{
    return assetmanageBLL._.Select(pageSize, pageIndex, where);
}
```

```
        public static IList<assetmanage> Select(int pageSize, int pageIndex,
string where, bool bParentTable, bool bChildrenTable)
        {
            return assetmanageBLL. _.Select(pageSize, pageIndex, where,
bParentTable, bChildrenTable);
        }

        public static IList<assetmanage> Select(CommandType commandType, string
sqlCommand, params Sqlconcontrol[] param)
        {
            return assetmanageBLL. _.Select(commandType, sqlCommand, param);
        }

        public static IList<assetmanage> Select(bool bParentTable, bool
bChildrenTable, CommandType commandType, string sqlCommand, params Sqlconcontrol[]
param)
        {
            return assetmanageBLL. _.Select(bParentTable, bChildrenTable,
commandType, sqlCommand, param);
        }

        public static bool Exists(int am_id)
        {
            return assetmanageBLL. _.Exists(am_id);
        }
    }
}

using System;
using System.Collections;
using System.controller.data;
using System.Data;
using System.Data.SqlClient;
using System.Text;
using controller.system;
using controller.system.I;
using controller.system.BLL;
namespace controller.system.SqlServer
{
    public partial class addresslist : Iaddresslist
    {
        public int Insert(addresslist addresslist)
        {
            string sqlCommand = "addresslistInsert";
            int res;
            Sqlconcontrol[] param = {
new Sqlconcontrol("@al_id", SqlDbType.Int),
new Sqlconcontrol("@al_u_id", SqlDbType.Int),
new Sqlconcontrol("@al_dept_id", SqlDbType.Int),
new Sqlconcontrol("@al_duty", SqlDbType.VarChar),
new Sqlconcontrol("@al_mobile", SqlDbType.VarChar),
new Sqlconcontrol("@al_smallclever", SqlDbType.VarChar),
new Sqlconcontrol("@al_worktelephone", SqlDbType.VarChar),
new Sqlconcontrol("@al_extension", SqlDbType.VarChar),
```



```

new Sqlconcontrol("@al_fax", SqlDbType.VarChar),
new Sqlconcontrol("@al_email", SqlDbType.VarChar),
new Sqlconcontrol("@al_qq", SqlDbType.VarChar),
new Sqlconcontrol("@al_msn", SqlDbType.VarChar),
new Sqlconcontrol("@al_address", SqlDbType.VarChar),
new Sqlconcontrol("@al_remark", SqlDbType.VarChar)
};

        param[0].Direction = concontrolDirection.Output;
        param[1].Value = addresslist.al_u_id;
        param[2].Value = addresslist.al_dept_id;
        param[3].Value = addresslist.al_duty;
        param[4].Value = addresslist.al_mobile;
        param[5].Value = addresslist.al_smallclever;
        param[6].Value = addresslist.al_worktelephone;
        param[7].Value = addresslist.al_extension;
        param[8].Value = addresslist.al_fax;
        param[9].Value = addresslist.al_email;
        param[10].Value = addresslist.al_qq;
        param[11].Value = addresslist.al_msn;
        param[12].Value = addresslist.al_address;
        param[13].Value = addresslist.al_remark;
        res = SqlHelper.ExecuteNonQuery(Conn.SqlConn,
CommandType.StoredProcedure, sqlCommand, param);
        addresslist.al_id = ((param[0].Value == DBNull.Value) ? 0 :
Convert.ToInt32(param[0].Value));
        return res;
    }

    public int Insert(SqlTransaction sp, addresslist addresslist)
    {
        string sqlCommand = "addresslistInsert";
        int res;
        Sqlconcontrol[] param = {
new Sqlconcontrol("@al_id", SqlDbType.Int),
new Sqlconcontrol("@al_u_id", SqlDbType.Int),
new Sqlconcontrol("@al_dept_id", SqlDbType.Int),
new Sqlconcontrol("@al_duty", SqlDbType.VarChar),
new Sqlconcontrol("@al_mobile", SqlDbType.VarChar),
new Sqlconcontrol("@al_smallclever", SqlDbType.VarChar),
new Sqlconcontrol("@al_worktelephone", SqlDbType.VarChar),
new Sqlconcontrol("@al_extension", SqlDbType.VarChar),
new Sqlconcontrol("@al_fax", SqlDbType.VarChar),
new Sqlconcontrol("@al_email", SqlDbType.VarChar),
new Sqlconcontrol("@al_qq", SqlDbType.VarChar),
new Sqlconcontrol("@al_msn", SqlDbType.VarChar),
new Sqlconcontrol("@al_address", SqlDbType.VarChar),
new Sqlconcontrol("@al_remark", SqlDbType.VarChar)
};

        param[0].Direction = concontrolDirection.Output;
        param[1].Value = addresslist.al_u_id;
        param[2].Value = addresslist.al_dept_id;

```

```
        param[3].Value = addresslist.al_duty;
        param[4].Value = addresslist.al_mobile;
        param[5].Value = addresslist.al_smallclever;
        param[6].Value = addresslist.al_worktelephone;
        param[7].Value = addresslist.al_extension;
        param[8].Value = addresslist.al_fax;
        param[9].Value = addresslist.al_email;
        param[10].Value = addresslist.al_qq;
        param[11].Value = addresslist.al_msn;
        param[12].Value = addresslist.al_address;
        param[13].Value = addresslist.al_remark;
        res = SqlHelper.ExecuteNonQuery(sp, CommandType.StoredProcedure,
sqlCommand, param);
        addresslist.al_id = ((param[0].Value) == DBNull.Value) ? 0 :
Convert.ToInt32(param[0].Value);
        return res;
    }

    public int Update(addresslist addresslist)
    {
        string sqlCommand = "addresslistUpdate";
        Sqlconcontrol[] param = {
new Sqlconcontrol("@al_id", SqlDbType.Int),
new Sqlconcontrol("@al_u_id", SqlDbType.Int),
new Sqlconcontrol("@al_dept_id", SqlDbType.Int),
new Sqlconcontrol("@al_duty", SqlDbType.VarChar),
new Sqlconcontrol("@al_mobile", SqlDbType.VarChar),
new Sqlconcontrol("@al_smallclever", SqlDbType.VarChar),
new Sqlconcontrol("@al_worktelephone", SqlDbType.VarChar),
new Sqlconcontrol("@al_extension", SqlDbType.VarChar),
new Sqlconcontrol("@al_fax", SqlDbType.VarChar),
new Sqlconcontrol("@al_email", SqlDbType.VarChar),
new Sqlconcontrol("@al_qq", SqlDbType.VarChar),
new Sqlconcontrol("@al_msn", SqlDbType.VarChar),
new Sqlconcontrol("@al_address", SqlDbType.VarChar),
new Sqlconcontrol("@al_remark", SqlDbType.VarChar)
};

        param[0].Value = addresslist.al_id;
        param[1].Value = addresslist.al_u_id;
        param[2].Value = addresslist.al_dept_id;
        param[3].Value = addresslist.al_duty;
        param[4].Value = addresslist.al_mobile;
        param[5].Value = addresslist.al_smallclever;
        param[6].Value = addresslist.al_worktelephone;
        param[7].Value = addresslist.al_extension;
        param[8].Value = addresslist.al_fax;
        param[9].Value = addresslist.al_email;
        param[10].Value = addresslist.al_qq;
        param[11].Value = addresslist.al_msn;
        param[12].Value = addresslist.al_address;
        param[13].Value = addresslist.al_remark;
```

```

        return SqlHelper.ExecuteNonQuery(Conn.SqlConn,
        CommandType.StoredProcedure, sqlCommand, param);
    }

    public int Update(SqlTransaction sp, addresslist addresslist)
    {
        string sqlCommand = "addresslistUpdate";
        Sqlconcontrol[] param = {
new Sqlconcontrol("@al_id", SqlDbType.Int),
new Sqlconcontrol("@al_u_id", SqlDbType.Int),
new Sqlconcontrol("@al_dept_id", SqlDbType.Int),
new Sqlconcontrol("@al_duty", SqlDbType.VarChar),
new Sqlconcontrol("@al_mobile", SqlDbType.VarChar),
new Sqlconcontrol("@al_smallclever", SqlDbType.VarChar),
new Sqlconcontrol("@al_worktelephone", SqlDbType.VarChar),
new Sqlconcontrol("@al_extension", SqlDbType.VarChar),
new Sqlconcontrol("@al_fax", SqlDbType.VarChar),
new Sqlconcontrol("@al_email", SqlDbType.VarChar),
new Sqlconcontrol("@al_qq", SqlDbType.VarChar),
new Sqlconcontrol("@al_msn", SqlDbType.VarChar),
new Sqlconcontrol("@al_address", SqlDbType.VarChar),
new Sqlconcontrol("@al_remark", SqlDbType.VarChar)
};

        param[0].Value = addresslist.al_id;
        param[1].Value = addresslist.al_u_id;
        param[2].Value = addresslist.al_dept_id;
        param[3].Value = addresslist.al_duty;
        param[4].Value = addresslist.al_mobile;
        param[5].Value = addresslist.al_smallclever;
        param[6].Value = addresslist.al_worktelephone;
        param[7].Value = addresslist.al_extension;
        param[8].Value = addresslist.al_fax;
        param[9].Value = addresslist.al_email;
        param[10].Value = addresslist.al_qq;
        param[11].Value = addresslist.al_msn;
        param[12].Value = addresslist.al_address;
        param[13].Value = addresslist.al_remark;
        return SqlHelper.ExecuteNonQuery(sp, CommandType.StoredProcedure,
sqlCommand, param);
    }

    public int Delete(int al_id)
    {
        string sqlCommand = "addresslistDelete";
        Sqlconcontrol[] param = {
new Sqlconcontrol("@al_id", SqlDbType.Int)
};

        param[0].Value = al_id;
        return SqlHelper.ExecuteNonQuery(Conn.SqlConn,
        CommandType.StoredProcedure, sqlCommand, param);
    }

    public int Delete(addresslist addresslist)

```

```

        {
            string sqlCommand = "addresslistDelete";
            Sqlconcontrol[] param = {
new Sqlconcontrol("@al_id", SqlDbType.Int)
};

            param[0].Value = addresslist.al_id;
            return
                SqlHelper.ExecuteNonQuery(Conn.SqlConn,
CommandType.StoredProcedure, sqlCommand, param);
        }

        public int Delete(SqlTransaction sp, int al_id)
        {
            string sqlCommand = "addresslistDelete";
            Sqlconcontrol[] param = {
new Sqlconcontrol("@al_id", SqlDbType.Int)
};

            param[0].Value = al_id;
            return SqlHelper.ExecuteNonQuery(sp, CommandType.StoredProcedure,
sqlCommand, param);
        }

        public int Delete(SqlTransaction sp, addresslist addresslist)
        {
            string sqlCommand = "addresslistDelete";
            Sqlconcontrol[] param = {
new Sqlconcontrol("@al_id", SqlDbType.Int)
};

            param[0].Value = addresslist.al_id;
            return SqlHelper.ExecuteNonQuery(sp, CommandType.StoredProcedure,
sqlCommand, param);
        }

        public addresslist Select(DataRow row)
        {
            addresslist obj = new addresslist();
            if (row != null)
            {
                obj.al_id = ((row["al_id"] == DBNull.Value) ? 0 :
Convert.ToInt32(row["al_id"]));
                obj.al_u_id = ((row["al_u_id"] == DBNull.Value) ? 0 :
Convert.ToInt32(row["al_u_id"]));
                obj.al_dept_id = ((row["al_dept_id"] == DBNull.Value) ? 0 :
Convert.ToInt32(row["al_dept_id"]));
                obj.al_duty = row["al_duty"].ToString();
                obj.al_mobile = row["al_mobile"].ToString();
                obj.al_smallclever = row["al_smallclever"].ToString();
                obj.al_worktelephone = row["al_worktelephone"].ToString();
                obj.al_extension = row["al_extension"].ToString();
                obj.al_fax = row["al_fax"].ToString();
                obj.al_email = row["al_email"].ToString();
                obj.al_qq = row["al_qq"].ToString();
                obj.al_msn = row["al_msn"].ToString();
                obj.al_address = row["al_address"].ToString();
            }
        }
    }

```

```

        obj.al_remark = row["al_remark"].ToString();
        obj.am_id = ((dr["am_id"]) == DBNull.Value) ? 0 :
Convert.ToInt32(dr["am_id"]);
        obj.am_assetname = dr["am_assetname"].ToString();
        obj.am_number = dr["am_number"].ToString();
        obj.am_measureunit = dr["am_measureunit"].ToString();
        obj.am_scalar = ((dr["am_scalar"]) == DBNull.Value) ? 0 :
Convert.ToInt32(dr["am_scalar"]);
        obj.am_spectype = dr["am_spectype"].ToString();
        obj.am_leavefactorynumber = dr["am_leavefactorynumber"].ToString();
        obj.am_artletternumber = dr["am_artletternumber"].ToString();
        obj.am_buyprice = ((dr["am_buyprice"]) == DBNull.Value) ? 0 :
Convert.ToDouble(dr["am_buyprice"]);
        obj.am_installcost = ((dr["am_installcost"]) == DBNull.Value) ? 0 :
Convert.ToDouble(dr["am_installcost"]);
        obj.am_depreciationcost = ((dr["am_depreciationcost"]) ==
DBNull.Value) ? 0 : Convert.ToDouble(dr["am_depreciationcost"]);
    }
    else
    {
        return null;
    }
    return obj;
}

public addresslist Select(IDataReader dr)
{
    addresslist obj = new addresslist();
    obj.al_id = ((dr["al_id"]) == DBNull.Value) ? 0 :
Convert.ToInt32(dr["al_id"]);
    obj.al_u_id = ((dr["al_u_id"]) == DBNull.Value) ? 0 :
Convert.ToInt32(dr["al_u_id"]);
    obj.al_dept_id = ((dr["al_dept_id"]) == DBNull.Value) ? 0 :
Convert.ToInt32(dr["al_dept_id"]);
    obj.al_duty = dr["al_duty"].ToString();
    obj.al_mobile = dr["al_mobile"].ToString();
    obj.al_smallclever = dr["al_smallclever"].ToString();
    obj.al_worktelephone = dr["al_worktelephone"].ToString();
    obj.al_extension = dr["al_extension"].ToString();
    obj.al_fax = dr["al_fax"].ToString();
    obj.al_email = dr["al_email"].ToString();
    obj.al_qq = dr["al_qq"].ToString();
    obj.al_msn = dr["al_msn"].ToString();
    obj.al_address = dr["al_address"].ToString();
    obj.al_remark = dr["al_remark"].ToString();
    return obj;
}

public addresslist Select(int al_id)
{
    return this.Select(al_id, false, false);
}

```

```

        public addresslist Select(int al_id, bool bParentTable, bool
bChildrenTable)
        {
            addresslist obj = null;
            Sqlconcontrol[] param = {
new Sqlconcontrol("@al_id", SqlDbType.Int)
};
            param[0].Value = al_id;
            string sqlCommand = "addresslistSelect";
            using (SqlDataReader dr = SqlHelper.ExecuteReader(Conn.SqlConn,
CommandType.StoredProcedure, sqlCommand, param))
            {
                while (dr.Read())
                {
                    obj = this.Select(dr);
                }
            }
            this.Select(obj, bParentTable, bChildrenTable);
            return obj;
        }
        public void Select(addresslist obj, bool bParentTable, bool
bChildrenTable)
        {
            if (bParentTable)
            {
                obj.users = usersBLL.Select(obj.al_u_id);
                obj.department = departmentBLL.Select(obj.al_dept_id);
            }
            if (bChildrenTable)
            {
            }
        }
        public void users(addresslist addresslist)
        {
            addresslist.users = usersBLL.Select(addresslist.al_u_id);
        }
        public void department(addresslist addresslist)
        {
            addresslist.department =
departmentBLL.Select(addresslist.al_dept_id);
        }
        public IList<addresslist> Select()
        {
            return this.Select(false, false);
        }
        public IList<addresslist> Select(bool bParentTable, bool bChildrenTable)
        {
            IList<addresslist> list = new List<addresslist>();
            string sqlCommand = "addresslistSelectAll";
            using (SqlDataReader dr = SqlHelper.ExecuteReader(Conn.SqlConn,

```

```
CommandType.StoredProcedure, sqlCommand))
    {
        while (dr.Read())
        {
            list.Add(this.Select(dr));
        }
    }
    foreach (addresslist obj in list)
    {
        this.Select(obj, bParentTable, bChildrenTable);
    }
    return list;
}

public IList<addresslist> Select(string where)
{
    return this.Select(where, false, false);
}

public IList<addresslist> Select(string where, bool bParentTable, bool
bChildrenTable)
{
    IList<addresslist> list = new List<addresslist>();
    Sqlconcontrol[] param = {
new Sqlconcontrol("@where", SqlDbType.VarChar, 8000)
};

    param[0].Value = where;
    string sqlCommand = "addresslistSelectByParams";
    using (SqlDataReader dr = SqlHelper.ExecuteReader(Conn.SqlConn,
CommandType.StoredProcedure, sqlCommand, param))
    {
        while (dr.Read())
        {
            list.Add(this.Select(dr));
        }
    }
    foreach (addresslist obj in list)
    {
        this.Select(obj, bParentTable, bChildrenTable);
    }
    return list;
}

public IList<addresslist> Select(string foreignFieldName, int
foreignFieldValue)
{
    return this.Select(foreignFieldName, foreignFieldValue, false,
false);
}

public IList<addresslist> Select(string foreignFieldName, int
foreignFieldValue, bool bParentTable, bool bChildrenTable)
{
    return this.Select(string.Format("{0}='{1}'", foreignFieldName,
```

```

foreignFieldValue), bParentTable, bChildrenTable);
    }
    public void Select(string where, out int recordCount)
    {
        string sqlCommand = "addresslistCountByWhere";
        Sqlconcontrol[] param = {
new Sqlconcontrol("@where", SqlDbType.VarChar, 8000),
new Sqlconcontrol("@recordCount", SqlDbType.Int)
};

        param[0].Value = where;
        param[1].Direction = concontrolDirection.Output;
        SqlHelper.ExecuteNonQuery(Conn.SqlConn, CommandType.StoredProcedure,
sqlCommand, param);
        recordCount = Convert.ToInt32(param[1].Value);
    }
    public IList<addresslist> Select(int pageSize, int pageIndex, string
where)
    {
        return this.Select(pageSize, pageIndex, where, false, false);
    }
    public IList<addresslist> Select(int pageSize, int pageIndex, string where,
bool bParentTable, bool bChildrenTable)
    {
        IList<addresslist> list = new List<addresslist>();
        string sqlCommand = "addresslistSelectByPagerParams";
        Sqlconcontrol[] param = {
new Sqlconcontrol("@pageSize", SqlDbType.Int),
new Sqlconcontrol("@pageIndex", SqlDbType.Int),
new Sqlconcontrol("@where", SqlDbType.VarChar, 8000)
};

        param[0].Value = pageSize;
        param[1].Value = pageIndex;
        param[2].Value = where;
        using (SqlDataReader dr = SqlHelper.ExecuteReader(Conn.SqlConn,
CommandType.StoredProcedure, sqlCommand, param))
        {
            while (dr.Read())
            {
                list.Add(this.Select(dr));
            }
        }
        foreach (addresslist obj in list)
        {
            this.Select(obj, bParentTable, bChildrenTable);
        }
        return list;
    }
    public IList<addresslist> Select(CommandType commandType, string
sqlCommand, params Sqlconcontrol[] param)
    {

```



```
        return this.Select(false, false, commandType, sqlCommand, param);
    }

    public IList<addresslist> Select(bool bParentTable, bool bChildrenTable,
    CommandType commandType, string sqlCommand, params Sqlconcontrol[] param)
    {
        IList<addresslist> list = new List<addresslist>();
        using (SqlDataReader dr = SqlHelper.ExecuteReader(Conn.SqlConn,
        commandType, sqlCommand, param))
        {
            while (dr.Read())
            {
                list.Add(this.Select(dr));
            }
        }
        foreach (addresslist obj in list)
        {
            this.Select(obj, bParentTable, bChildrenTable);
        }
        return list;
    }

    public bool Exists(int al_id)
    {
        Sqlconcontrol[] param = {
            new Sqlconcontrol("@al_id", SqlDbType.Int)
        };
        param[0].Value = al_id;
        string sqlCommand = "addresslistIsExist";
        int a = Convert.ToInt32(SqlHelper.ExecuteScalar(Conn.SqlConn,
        CommandType.StoredProcedure, sqlCommand, param));
        if (a > 0)
        {
            return true;
        }
        else
        {
            return false;
        }
    }
}

using System;
using System.Collections;
using System.controller.data;
using System.Data;
using System.Data.SqlClient;
using System.Text;
using controller.system;
using controller.system;
using controller.system.BLL;
namespace controller.system.SqlServer
```

```

{
    public partial class assetmanage : Iassetmanage
    {
        public int Insert(assetmanage assetmanage)
        {
            string sqlCommand = "assetmanageInsert";
            int res;
            Sqlconcontrol[] param = {
new Sqlconcontrol("@am_id", SqlDbType.Int),
new Sqlconcontrol("@am_assetname", SqlDbType.VarChar),
new Sqlconcontrol("@am_number", SqlDbType.VarChar),
new Sqlconcontrol("@am_measureunit", SqlDbType.VarChar),
new Sqlconcontrol("@am_scalar", SqlDbType.Int),
new Sqlconcontrol("@am_spectype", SqlDbType.VarChar),
new Sqlconcontrol("@am_leavefactorynumber", SqlDbType.VarChar),
new Sqlconcontrol("@am_artletternumber", SqlDbType.VarChar),
new Sqlconcontrol("@am_buyprice", SqlDbType.Float),
new Sqlconcontrol("@am_installcost", SqlDbType.Float),
new Sqlconcontrol("@am_depreciationcost", SqlDbType.Float),
new Sqlconcontrol("@am_betvalue", SqlDbType.Float),
new Sqlconcontrol("@am_production", SqlDbType.VarChar),
new Sqlconcontrol("@am_productiondate", SqlDbType.DateTime),
new Sqlconcontrol("@am_deliverusetime", SqlDbType.DateTime),
new Sqlconcontrol("@am_advanceuseyear", SqlDbType.Int),
new Sqlconcontrol("@am_buyordept", SqlDbType.VarChar),
new Sqlconcontrol("@am_storageordept", SqlDbType.VarChar),
new Sqlconcontrol("@am_purpose", SqlDbType.VarChar),
new Sqlconcontrol("@am_dept_id", SqlDbType.Int),
new Sqlconcontrol("@am_useaddress", SqlDbType.VarChar),
new Sqlconcontrol("@am_ustate", SqlDbType.Int),
new Sqlconcontrol("@am_bookerid", SqlDbType.Int),
new Sqlconcontrol("@am_remark", SqlDbType.VarChar),
new Sqlconcontrol("@am_registertime", SqlDbType.DateTime),
new Sqlconcontrol("@am_isscrap", SqlDbType.Int)
};

            param[0].Direction = ParameterDirection.Output;
            param[1].Value = assetmanage.am_assetname;
            param[2].Value = assetmanage.am_number;
            param[3].Value = assetmanage.am_measureunit;
            param[4].Value = assetmanage.am_scalar;
            param[5].Value = assetmanage.am_spectype;
            param[6].Value = assetmanage.am_leavefactorynumber;
            param[7].Value = assetmanage.am_artletternumber;
            param[8].Value = assetmanage.am_buyprice;
            param[9].Value = assetmanage.am_installcost;
            param[10].Value = assetmanage.am_depreciationcost;
            param[11].Value = assetmanage.am_betvalue;
            param[12].Value = assetmanage.am_production;
            param[13].Value = assetmanage.am_productiondate;
            param[14].Value = assetmanage.am_deliverusetime;

```

```

        param[15].Value = assetmanage.am_advanceuseyear;
        param[16].Value = assetmanage.am_buyordept;
        param[17].Value = assetmanage.am_storageordept;
        param[18].Value = assetmanage.am_purpose;
        param[19].Value = assetmanage.am_dept_id;
        param[20].Value = assetmanage.am_useaddress;
        param[21].Value = assetmanage.am_usestate;
        param[22].Value = assetmanage.am_bookerid;
        param[23].Value = assetmanage.am_remark;
        param[24].Value = assetmanage.am_registertime;
        param[25].Value = assetmanage.am_isscrap;
        res = SqlHelper.ExecuteNonQuery(Conn.SqlConn,
        CommandType.StoredProcedure, sqlCommand, param);
        assetmanage.am_id = ((param[0].Value == DBNull.Value) ? 0 :
        Convert.ToInt32(param[0].Value));
        return res;
    }

    public int Insert(SqlTransaction sp, assetmanage assetmanage)
    {
        string sqlCommand = "assetmanageInsert";
        int res;
        Sqlconcontrol[] param = {
new Sqlconcontrol("@am_id", SqlDbType.Int),
new Sqlconcontrol("@am_assetname", SqlDbType.VarChar),
new Sqlconcontrol("@am_number", SqlDbType.VarChar),
new Sqlconcontrol("@am_measureunit", SqlDbType.VarChar),
new Sqlconcontrol("@am_scalar", SqlDbType.Int),
new Sqlconcontrol("@am_spectype", SqlDbType.VarChar),
new Sqlconcontrol("@am_leavefactorynumber", SqlDbType.VarChar),
new Sqlconcontrol("@am_artletternumber", SqlDbType.VarChar),
new Sqlconcontrol("@am_buyprice", SqlDbType.Flcontroller.systemt),
new Sqlconcontrol("@am_installcost", SqlDbType.Flcontroller.systemt),
new Sqlconcontrol("@am_depreciationcost", SqlDbType.Flcontroller.systemt),
new Sqlconcontrol("@am_betvalue", SqlDbType.Flcontroller.systemt),
new Sqlconcontrol("@am_production", SqlDbType.VarChar),
new Sqlconcontrol("@am_productiondate", SqlDbType.DateTime),
new Sqlconcontrol("@am_deliverusetime", SqlDbType.DateTime),
new Sqlconcontrol("@am_advanceuseyear", SqlDbType.Int),
new Sqlconcontrol("@am_buyordept", SqlDbType.VarChar),
new Sqlconcontrol("@am_storageordept", SqlDbType.VarChar),
new Sqlconcontrol("@am_purpose", SqlDbType.VarChar),
new Sqlconcontrol("@am_dept_id", SqlDbType.Int),
new Sqlconcontrol("@am_useaddress", SqlDbType.VarChar),
new Sqlconcontrol("@am_usestate", SqlDbType.Int),
new Sqlconcontrol("@am_bookerid", SqlDbType.Int),
new Sqlconcontrol("@am_remark", SqlDbType.VarChar),
new Sqlconcontrol("@am_registertime", SqlDbType.DateTime),
new Sqlconcontrol("@am_isscrap", SqlDbType.Int)
        };
        param[0].Direction = concontrolDirection.Output;
    }

```

```

        param[1].Value = assetmanage.am_assetname;
        param[2].Value = assetmanage.am_number;
        param[3].Value = assetmanage.am_measureunit;
        param[4].Value = assetmanage.am_scalar;
        param[5].Value = assetmanage.am_spectype;
        param[6].Value = assetmanage.am_leavefactorynumber;
        param[7].Value = assetmanage.am_artletternumber;
        param[8].Value = assetmanage.am_buyprice;
        param[9].Value = assetmanage.am_installcost;
        param[10].Value = assetmanage.am_depreciationcost;
        param[11].Value = assetmanage.am_betvalue;
        param[12].Value = assetmanage.am_production;
        param[13].Value = assetmanage.am_productiondate;
        param[14].Value = assetmanage.am_deliverusetime;
        param[15].Value = assetmanage.am_advanceuseyear;
        param[16].Value = assetmanage.am_buyordept;
        param[17].Value = assetmanage.am_storageordept;
        param[18].Value = assetmanage.am_purpose;
        param[19].Value = assetmanage.am_dept_id;
        param[20].Value = assetmanage.am_useaddress;
        param[21].Value = assetmanage.am_usestate;
        param[22].Value = assetmanage.am_bookerid;
        param[23].Value = assetmanage.am_remark;
        param[24].Value = assetmanage.am_registertime;
        param[25].Value = assetmanage.am_isscrap;
        res = SqlHelper.ExecuteNonQuery(sp, CommandType.StoredProcedure,
sqlCommand, param);
        assetmanage.am_id = ((param[0].Value) == DBNull.Value) ? 0 :
Convert.ToInt32(param[0].Value);
        return res;
    }

    public int Update(assetmanage assetmanage)
    {
        string sqlCommand = "assetmanageUpdate";
        Sqlconcontrol[] param = {
new Sqlconcontrol("@am_id", SqlDbType.Int),
new Sqlconcontrol("@am_assetname", SqlDbType.VarChar),
new Sqlconcontrol("@am_number", SqlDbType.VarChar),
new Sqlconcontrol("@am_measureunit", SqlDbType.VarChar),
new Sqlconcontrol("@am_scalar", SqlDbType.Int),
new Sqlconcontrol("@am_spectype", SqlDbType.VarChar),
new Sqlconcontrol("@am_leavefactorynumber", SqlDbType.VarChar),
new Sqlconcontrol("@am_artletternumber", SqlDbType.VarChar),
new Sqlconcontrol("@am_buyprice", SqlDbType.Flcontroller.systemt),
new Sqlconcontrol("@am_installcost", SqlDbType.Flcontroller.systemt),
new Sqlconcontrol("@am_depreciationcost", SqlDbType.Flcontroller.systemt),
new Sqlconcontrol("@am_betvalue", SqlDbType.Flcontroller.systemt),
new Sqlconcontrol("@am_production", SqlDbType.VarChar),
new Sqlconcontrol("@am_productiondate", SqlDbType.DateTime),
new Sqlconcontrol("@am_deliverusetime", SqlDbType.DateTime),

```

```

new Sqlconcontrol("@am_advanceuseyear", SqlDbType.Int),
new Sqlconcontrol("@am_buyordept", SqlDbType.VarChar),
new Sqlconcontrol("@am_storageordept", SqlDbType.VarChar),
new Sqlconcontrol("@am_purpose", SqlDbType.VarChar),
new Sqlconcontrol("@am_dept_id", SqlDbType.Int),
new Sqlconcontrol("@am_useaddress", SqlDbType.VarChar),
new Sqlconcontrol("@am_ustate", SqlDbType.Int),
new Sqlconcontrol("@am_bookerid", SqlDbType.Int),
new Sqlconcontrol("@am_remark", SqlDbType.VarChar),
new Sqlconcontrol("@am_registertime", SqlDbType.DateTime),
new Sqlconcontrol("@am_isscrap", SqlDbType.Int)
};

        param[0].Value = assetmanage.am_id;
        param[1].Value = assetmanage.am_assetname;
        param[2].Value = assetmanage.am_number;
        param[3].Value = assetmanage.am_measureunit;
        param[4].Value = assetmanage.am_scalar;
        param[5].Value = assetmanage.am_spectype;
        param[6].Value = assetmanage.am_leavefactorynumber;
        param[7].Value = assetmanage.am_artletternumber;
        param[8].Value = assetmanage.am_buyprice;
        param[9].Value = assetmanage.am_installcost;
        param[10].Value = assetmanage.am_depreciationcost;
        param[11].Value = assetmanage.am_betvalue;
        param[12].Value = assetmanage.am_production;
        param[13].Value = assetmanage.am_productiondate;
        param[14].Value = assetmanage.am_deliverustime;
        param[15].Value = assetmanage.am_advanceuseyear;
        param[16].Value = assetmanage.am_buyordept;
        param[17].Value = assetmanage.am_storageordept;
        param[18].Value = assetmanage.am_purpose;
        param[19].Value = assetmanage.am_dept_id;
        param[20].Value = assetmanage.am_useaddress;
        param[21].Value = assetmanage.am_ustate;
        param[22].Value = assetmanage.am_bookerid;
        param[23].Value = assetmanage.am_remark;
        param[24].Value = assetmanage.am_registertime;
        param[25].Value = assetmanage.am_isscrap;
        return SqlHelper.ExecuteNonQuery(Conn.SqlConn,
CommandType.StoredProcedure, sqlCommand, param);
    }

    public int Update(SqlTransaction sp, assetmanage assetmanage)
    {
        string sqlCommand = "assetmanageUpdate";
        Sqlconcontrol[] param = {
new Sqlconcontrol("@am_id", SqlDbType.Int),
new Sqlconcontrol("@am_assetname", SqlDbType.VarChar),
new Sqlconcontrol("@am_number", SqlDbType.VarChar),
new Sqlconcontrol("@am_measureunit", SqlDbType.VarChar),
new Sqlconcontrol("@am_scalar", SqlDbType.Int),

```

```
new Sqlconcontrol("@am_spectype", SqlDbType.VarChar),
new Sqlconcontrol("@am_leavefactorynumber", SqlDbType.VarChar),
new Sqlconcontrol("@am_artletternumber", SqlDbType.VarChar),
new Sqlconcontrol("@am_buyprice", SqlDbType.Float),
new Sqlconcontrol("@am_installcost", SqlDbType.Float),
new Sqlconcontrol("@am_depreciationcost", SqlDbType.Float),
new Sqlconcontrol("@am_betvalue", SqlDbType.Float),
new Sqlconcontrol("@am_production", SqlDbType.VarChar),
new Sqlconcontrol("@am_productiondate", SqlDbType.DateTime),
new Sqlconcontrol("@am_deliverusetime", SqlDbType.DateTime),
new Sqlconcontrol("@am_advanceuseyear", SqlDbType.Int),
new Sqlconcontrol("@am_buyordept", SqlDbType.VarChar),
new Sqlconcontrol("@am_storageordept", SqlDbType.VarChar),
new Sqlconcontrol("@am_purpose", SqlDbType.VarChar),
new Sqlconcontrol("@am_dept_id", SqlDbType.Int),
new Sqlconcontrol("@am_useaddress", SqlDbType.VarChar),
new Sqlconcontrol("@am_ustate", SqlDbType.Int),
new Sqlconcontrol("@am_bookerid", SqlDbType.Int),
new Sqlconcontrol("@am_remark", SqlDbType.VarChar),
new Sqlconcontrol("@am_registertime", SqlDbType.DateTime),
new Sqlconcontrol("@am_isscrap", SqlDbType.Int)
};

param[0].Value = assetmanage.am_id;
param[1].Value = assetmanage.am_assetname;
param[2].Value = assetmanage.am_number;
param[3].Value = assetmanage.am_measureunit;
param[4].Value = assetmanage.am_scalar;
param[5].Value = assetmanage.am_spectype;
param[6].Value = assetmanage.am_leavefactorynumber;
param[7].Value = assetmanage.am_artletternumber;
param[8].Value = assetmanage.am_buyprice;
param[9].Value = assetmanage.am_installcost;
param[10].Value = assetmanage.am_depreciationcost;
param[11].Value = assetmanage.am_betvalue;
param[12].Value = assetmanage.am_production;
param[13].Value = assetmanage.am_productiondate;
param[14].Value = assetmanage.am_deliverusetime;
param[15].Value = assetmanage.am_advanceuseyear;
param[16].Value = assetmanage.am_buyordept;
param[17].Value = assetmanage.am_storageordept;
param[18].Value = assetmanage.am_purpose;
param[19].Value = assetmanage.am_dept_id;
param[20].Value = assetmanage.am_useaddress;
param[21].Value = assetmanage.am_ustate;
param[22].Value = assetmanage.am_bookerid;
param[23].Value = assetmanage.am_remark;
param[24].Value = assetmanage.am_registertime;
param[25].Value = assetmanage.am_isscrap;
return SqlHelper.ExecuteNonQuery(sp, CommandType.StoredProcedure,
sqlCommand, param);
```

```

    }
    public int Delete(int am_id)
    {
        string sqlCommand = "assetmanageDelete";
        Sqlconcontrol[] param = {
new Sqlconcontrol("@am_id", SqlDbType.Int)
};
        param[0].Value = am_id;
        return
            SqlHelper.ExecuteNonQuery(Conn.SqlConn,
CommandType.StoredProcedure, sqlCommand, param);
    }
    public int Delete(assetmanage assetmanage)
    {
        string sqlCommand = "assetmanageDelete";
        Sqlconcontrol[] param = {
new Sqlconcontrol("@am_id", SqlDbType.Int)
};
        param[0].Value = assetmanage.am_id;
        return
            SqlHelper.ExecuteNonQuery(Conn.SqlConn,
CommandType.StoredProcedure, sqlCommand, param);
    }
    public int Delete(SqlTransaction sp, int am_id)
    {
        string sqlCommand = "assetmanageDelete";
        Sqlconcontrol[] param = {
new Sqlconcontrol("@am_id", SqlDbType.Int)
};
        param[0].Value = am_id;
        return
            SqlHelper.ExecuteNonQuery(sp, CommandType.StoredProcedure,
sqlCommand, param);
    }
    public int Delete(SqlTransaction sp, assetmanage assetmanage)
    {
        string sqlCommand = "assetmanageDelete";
        Sqlconcontrol[] param = {
new Sqlconcontrol("@am_id", SqlDbType.Int)
};
        param[0].Value = assetmanage.am_id;
        return
            SqlHelper.ExecuteNonQuery(sp, CommandType.StoredProcedure,
sqlCommand, param);
    }
    public assetmanage Select(DataRow row)
    {
        assetmanage obj = new assetmanage();
        if (row != null)
        {
            obj.am_id = ((row["am_id"]) == DBNull.Value) ? 0 :
Convert.ToInt32(row["am_id"]);
            obj.am_assetname = row["am_assetname"].ToString();
            obj.am_number = row["am_number"].ToString();

```

```

        obj.am_measureunit = row["am_measureunit"].ToString();
        obj.am_scalar = ((row["am_scalar"] == DBNull.Value) ? 0 :
Convert.ToInt32(row["am_scalar"]));
        obj.am_spectype = row["am_spectype"].ToString();
        obj.am_leavefactorynumber =
row["am_leavefactorynumber"].ToString();
        obj.am_artletternumber = row["am_artletternumber"].ToString();
        obj.am_buyprice = ((row["am_buyprice"] == DBNull.Value) ? 0 :
Convert.ToDouble(row["am_buyprice"]));
        obj.am_installcost = ((row["am_installcost"] == DBNull.Value) ?
0 : Convert.ToDouble(row["am_installcost"]));
        obj.am_depreciationcost = ((row["am_depreciationcost"] ==
DBNull.Value) ? 0 : Convert.ToDouble(row["am_depreciationcost"]));
        obj.am_betvalue = ((row["am_betvalue"] == DBNull.Value) ? 0 :
Convert.ToDouble(row["am_betvalue"]));
        obj.am_production = row["am_production"].ToString();
        obj.am_productiondate = ((row["am_productiondate"] ==
DBNull.Value) ? DateTime.MinValue : Convert.ToDateTime(row["am_productiondate"]));
        obj.am_deliverusetime = ((row["am_deliverusetime"] ==
DBNull.Value) ? DateTime.MinValue : Convert.ToDateTime(row["am_deliverusetime"]));
        obj.am_advanceuseyear = ((row["am_advanceuseyear"] ==
DBNull.Value) ? 0 : Convert.ToInt32(row["am_advanceuseyear"]));
        obj.am_buyordept = row["am_buyordept"].ToString();
        obj.am_storageordept = row["am_storageordept"].ToString();
        obj.am_purpose = row["am_purpose"].ToString();
        obj.am_dept_id = ((row["am_dept_id"] == DBNull.Value) ? 0 :
Convert.ToInt32(row["am_dept_id"]));
        obj.am_useaddress = row["am_useaddress"].ToString();
        obj.am_usestate = ((row["am_usestate"] == DBNull.Value) ? 0 :
Convert.ToInt32(row["am_usestate"]));
        obj.am_bookerid = ((row["am_bookerid"] == DBNull.Value) ? 0 :
Convert.ToInt32(row["am_bookerid"]));
        obj.am_remark = row["am_remark"].ToString();
        obj.am_registertime = ((row["am_registertime"] == DBNull.Value) ?
DateTime.MinValue : Convert.ToDateTime(row["am_registertime"]));
        obj.am_isscrap = ((row["am_isscrap"] == DBNull.Value) ? 0 :
Convert.ToInt32(row["am_isscrap"]));
    }
    else
    {
        return null;
    }
    return obj;
}

public assetmanage Select(IDataReader dr)
{
    assetmanage obj = new assetmanage();
    obj.am_id = ((dr["am_id"] == DBNull.Value) ? 0 :
Convert.ToInt32(dr["am_id"]));
    obj.am_assetname = dr["am_assetname"].ToString();

```



```

        obj.am_number = dr["am_number"].ToString();
        obj.am_measureunit = dr["am_measureunit"].ToString();
        obj.am_scalar = ((dr["am_scalar"] == DBNull.Value) ? 0 :
Convert.ToInt32(dr["am_scalar"]));
        obj.am_spectype = dr["am_spectype"].ToString();
        obj.am_leavefactorynumber = dr["am_leavefactorynumber"].ToString();
        obj.am_artletternumber = dr["am_artletternumber"].ToString();
        obj.am_buyprice = ((dr["am_buyprice"] == DBNull.Value) ? 0 :
Convert.ToDouble(dr["am_buyprice"]));
        obj.am_installcost = ((dr["am_installcost"] == DBNull.Value) ? 0 :
Convert.ToDouble(dr["am_installcost"]));
        obj.am_depreciationcost = ((dr["am_depreciationcost"] ==
DBNull.Value) ? 0 : Convert.ToDouble(dr["am_depreciationcost"]));
        obj.am_betvalue = ((dr["am_betvalue"] == DBNull.Value) ? 0 :
Convert.ToDouble(dr["am_betvalue"]));
        obj.am_production = dr["am_production"].ToString();
        obj.am_productiondate = ((dr["am_productiondate"] == DBNull.Value) ?
DateTime.MinValue : Convert.ToDateTime(dr["am_productiondate"]));
        obj.am_deliverusetime = ((dr["am_deliverusetime"] == DBNull.Value) ?
DateTime.MinValue : Convert.ToDateTime(dr["am_deliverusetime"]));
        obj.am_advanceuseyear = ((dr["am_advanceuseyear"] == DBNull.Value) ?
0 : Convert.ToInt32(dr["am_advanceuseyear"]));
        obj.am_buyordept = dr["am_buyordept"].ToString();
        obj.am_storageordept = dr["am_storageordept"].ToString();
        obj.am_purpose = dr["am_purpose"].ToString();
        obj.am_dept_id = ((dr["am_dept_id"] == DBNull.Value) ? 0 :
Convert.ToInt32(dr["am_dept_id"]));
        obj.am_useaddress = dr["am_useaddress"].ToString();
        obj.am_ustate = ((dr["am_ustate"] == DBNull.Value) ? 0 :
Convert.ToInt32(dr["am_ustate"]));
        obj.am_bookerid = ((dr["am_bookerid"] == DBNull.Value) ? 0 :
Convert.ToInt32(dr["am_bookerid"]));
        obj.am_remark = dr["am_remark"].ToString();
        obj.am_registertime = ((dr["am_registertime"] == DBNull.Value) ?
DateTime.MinValue : Convert.ToDateTime(dr["am_registertime"]));
        obj.am_isscrap = ((dr["am_isscrap"] == DBNull.Value) ? 0 :
Convert.ToInt32(dr["am_isscrap"]));
        return obj;
    }

    public assetmanage Select(int am_id)
    {
        return this.Select(am_id, false, false);
    }

    public assetmanage Select(int am_id, bool bParentTable, bool
bChildrenTable)
    {
        assetmanage obj = null;
        Sqlconcontrol[] param = {
new Sqlconcontrol("@am_id", SqlDbType.Int)
};

```

```

        param[0].Value = am_id;
        string sqlCommand = "assetmanageSelect";
        using (SqlDataReader dr = SqlHelper.ExecuteReader(Conn.SqlConn,
CommandType.StoredProcedure, sqlCommand, param))
        {
            while (dr.Read())
            {
                obj = this.Select(dr);
            }
        }
        this.Select(obj, bParentTable, bChildrenTable);
        return obj;
    }

    public void Select(assetmanage obj, bool bParentTable, bool
bChildrenTable)
    {
        if (bParentTable)
        {
            obj.department = departmentBLL.Select(obj.am_dept_id);
        }
        if (bChildrenTable)
        {
        }
    }

    public void department(assetmanage assetmanage)
    {
        new Sqlconcontrol("@att_id", SqlDbType.Int),
new Sqlconcontrol("@att_date", SqlDbType.DateTime),
new Sqlconcontrol("@att_forenoontime", SqlDbType.DateTime),
new Sqlconcontrol("@att_afternoontime", SqlDbType.DateTime),
new Sqlconcontrol("@att_u_id", SqlDbType.Int),
new Sqlconcontrol("@att_dept_id", SqlDbType.Int)
        assetmanage.department =
departmentBLL.Select(assetmanage.am_dept_id);
    }

    public IList<assetmanage> Select()
    {
        return this.Select(false, false);
    }

    public IList<assetmanage> Select(bool bParentTable, bool bChildrenTable)
    {
        IList<assetmanage> list = new List<assetmanage>();
        string sqlCommand = "assetmanageSelectAll";
        using (SqlDataReader dr = SqlHelper.ExecuteReader(Conn.SqlConn,
CommandType.StoredProcedure, sqlCommand))
        {
            while (dr.Read())
            {
                list.Add(this.Select(dr));
            }
        }
    }

```

```

    }
    foreach (assetmanage obj in list)
    {
        this.Select(obj, bParentTable, bChildrenTable);
    }
    return list;
}

public IList<assetmanage> Select(string where)
{
    return this.Select(where, false, false);
}

public IList<assetmanage> Select(string where, bool bParentTable, bool
bChildrenTable)
{
    IList<assetmanage> list = new List<assetmanage>();
    Sqlconcontrol[] param ={
new Sqlconcontrol("@where", SqlDbType.VarChar, 8000)
};

    param[0].Value = where;
    string sqlCommand = "assetmanageSelectByParams";
    using (SqlDataReader dr = SqlHelper.ExecuteReader(Conn.SqlConn,
CommandType.StoredProcedure, sqlCommand, param))
    {
        while (dr.Read())
        {
            list.Add(this.Select(dr));
        }
    }
    foreach (assetmanage obj in list)
    {
        this.Select(obj, bParentTable, bChildrenTable);
    }
    return list;
}

public IList<assetmanage> Select(string foreignFieldName, int
foreignFieldValue)
{
    return this.Select(foreignFieldName, foreignFieldValue, false,
false);
}

public IList<assetmanage> Select(string foreignFieldName, int
foreignFieldValue, bool bParentTable, bool bChildrenTable)
{
    return this.Select(string.Format("{0}='{1}'", foreignFieldName,
foreignFieldValue), bParentTable, bChildrenTable);
}

public void Select(string where, out int recordCount)
{
    string sqlCommand = "assetmanageCountByWhere";
    Sqlconcontrol[] param ={

```

```

new SqlParameter("@where", SqlDbType.VarChar, 8000),
new SqlParameter("@recordCount", SqlDbType.Int)
};

        param[0].Value = where;
        param[1].Direction = ParameterDirection.Output;
        SqlHelper.ExecuteNonQuery(Conn.SqlConn, CommandType.StoredProcedure,
sqlCommand, param);
        recordCount = Convert.ToInt32(param[1].Value);
    }
    public IList<assetmanage> Select(int pageSize, int pageIndex, string
where)
    {
        return this.Select(pageSize, pageIndex, where, false, false);
    }
    public IList<assetmanage> Select(int pageSize, int pageIndex, string where,
bool bParentTable, bool bChildrenTable)
    {
        IList<assetmanage> list = new List<assetmanage>();
        string sqlCommand = "assetmanageSelectByPagerParams";
        SqlParameter[] param = {
new SqlParameter("@pageSize", SqlDbType.Int),
new SqlParameter("@pageIndex", SqlDbType.Int),
new SqlParameter("@where", SqlDbType.VarChar, 8000)
};

        param[0].Value = pageSize;
        param[1].Value = pageIndex;
        param[2].Value = where;
        using (SqlDataReader dr = SqlHelper.ExecuteReader(Conn.SqlConn,
CommandType.StoredProcedure, sqlCommand, param))
        {
            while (dr.Read())
            {
                list.Add(this.Select(dr));
            }
        }
        foreach (assetmanage obj in list)
        {
            this.Select(obj, bParentTable, bChildrenTable);
        }
        return list;
    }
    public IList<assetmanage> Select(CommandType commandType, string
sqlCommand, params SqlParameter[] param)
    {
        return this.Select(false, false, commandType, sqlCommand, param);
    }
    public IList<assetmanage> Select(bool bParentTable, bool bChildrenTable,
CommandType commandType, string sqlCommand, params SqlParameter[] param)
    {
        IList<assetmanage> list = new List<assetmanage>();

```

```
        using (SqlDataReader dr = SqlHelper.ExecuteReader(Conn.SqlConn,
commandType, sqlCommand, param))
        {
            while (dr.Read())
            {
                list.Add(this.Select(dr));
            }
        }
        foreach (assetmanage obj in list)
        {
            this.Select(obj, bParentTable, bChildrenTable);
        }
        return list;
    }
    public bool Exists(int am_id)
    {
        Sqlconcontrol[] param = {
            new Sqlconcontrol("@am_id", SqlDbType.Int)
        };
        param[0].Value = am_id;
        string sqlCommand = "assetmanageIsExist";
        int a = Convert.ToInt32(SqlHelper.ExecuteScalar(Conn.SqlConn,
CommandType.StoredProcedure, sqlCommand, param));
        if (a > 0)
        {
            return true;
        }
        else
        {
            return false;
        }
    }
}

using System;
using System.Collections;
using System.controller.data;
using System.Data;
using System.Data.SqlClient;
using System.Text;
using controller.system;
using controller.system.I;
using controller.system.BLL;
namespace controller.system.SqlServer
{
    public partial class attendance : Iattendance
    {
        public int Insert(attendance attendance)
        {
            string sqlCommand = "attendanceInsert";
```

```

        int res;
        Sqlconcontrol[] param = {
new Sqlconcontrol("@att_id", SqlDbType.Int),
new Sqlconcontrol("@att_date", SqlDbType.DateTime),
new Sqlconcontrol("@att_forenoontime", SqlDbType.DateTime),
new Sqlconcontrol("@att_afternoontime", SqlDbType.DateTime),
new Sqlconcontrol("@att_u_id", SqlDbType.Int),
new Sqlconcontrol("@att_dept_id", SqlDbType.Int)
};

        param[0].Direction = concontrolDirection.Output;
        param[1].Value = attendance.att_date;
        param[2].Value = attendance.att_forenoontime;
        param[3].Value = attendance.att_afternoontime;
        param[4].Value = attendance.att_u_id;
        param[5].Value = attendance.att_dept_id;
        res = SqlHelper.ExecuteNonQuery(Conn.SqlConn,
CommandType.StoredProcedure, sqlCommand, param);
        attendance.att_id = ((param[0].Value == DBNull.Value) ? 0 :
Convert.ToInt32(param[0].Value));
        return res;
    }

    public int Insert(SqlTransaction sp, attendance attendance)
    {
        string sqlCommand = "attendanceInsert";
        int res;
        Sqlconcontrol[] param = {
new Sqlconcontrol("@att_id", SqlDbType.Int),
new Sqlconcontrol("@att_date", SqlDbType.DateTime),
new Sqlconcontrol("@att_forenoontime", SqlDbType.DateTime),
new Sqlconcontrol("@att_afternoontime", SqlDbType.DateTime),
new Sqlconcontrol("@att_u_id", SqlDbType.Int),
new Sqlconcontrol("@att_dept_id", SqlDbType.Int)
};

        param[0].Direction = concontrolDirection.Output;
        param[1].Value = attendance.att_date;
        param[2].Value = attendance.att_forenoontime;
        param[3].Value = attendance.att_afternoontime;
        param[4].Value = attendance.att_u_id;
        param[5].Value = attendance.att_dept_id;
        res = SqlHelper.ExecuteNonQuery(sp, CommandType.StoredProcedure,
sqlCommand, param);
        attendance.att_id = ((param[0].Value == DBNull.Value) ? 0 :
Convert.ToInt32(param[0].Value));
        return res;
    }

    public int Update(attendance attendance)
    {
        string sqlCommand = "attendanceUpdate";
        Sqlconcontrol[] param = {
new Sqlconcontrol("@att_id", SqlDbType.Int),

```

```

new Sqlconcontrol("@att_date", SqlDbType.DateTime),
new Sqlconcontrol("@att_forenoontime", SqlDbType.DateTime),
new Sqlconcontrol("@att_afternoontime", SqlDbType.DateTime),
new Sqlconcontrol("@att_u_id", SqlDbType.Int),
new Sqlconcontrol("@att_dept_id", SqlDbType.Int)
};

        param[0].Value = attendance.att_id;
        param[1].Value = attendance.att_date;
        param[2].Value = attendance.att_forenoontime;
        param[3].Value = attendance.att_afternoontime;
        param[4].Value = attendance.att_u_id;
        param[5].Value = attendance.att_dept_id;
        return
            SqlHelper.ExecuteNonQuery(Conn. SqlConnection,
CommandType.StoredProcedure, sqlCommand, param);
    }

    public int Update(SqlTransaction sp, attendance attendance)
    {
        string sqlCommand = "attendanceUpdate";
        Sqlconcontrol[] param = {
new Sqlconcontrol("@att_id", SqlDbType.Int),
new Sqlconcontrol("@att_date", SqlDbType.DateTime),
new Sqlconcontrol("@att_forenoontime", SqlDbType.DateTime),
new Sqlconcontrol("@att_afternoontime", SqlDbType.DateTime),
new Sqlconcontrol("@att_u_id", SqlDbType.Int),
new Sqlconcontrol("@att_dept_id", SqlDbType.Int)
};

        param[0].Value = attendance.att_id;
        param[1].Value = attendance.att_date;
        param[2].Value = attendance.att_forenoontime;
        param[3].Value = attendance.att_afternoontime;
        param[4].Value = attendance.att_u_id;
        param[5].Value = attendance.att_dept_id;
        return SqlHelper.ExecuteNonQuery(sp, CommandType.StoredProcedure,
sqlCommand, param);
    }

    public int Delete(int att_id)
    {
        string sqlCommand = "attendanceDelete";
        Sqlconcontrol[] param = {
new Sqlconcontrol("@att_id", SqlDbType.Int)
};

        param[0].Value = att_id;
        return
            SqlHelper.ExecuteNonQuery(Conn. SqlConnection,
CommandType.StoredProcedure, sqlCommand, param);
    }

    m name="attendance">attendance</param>
    public int Delete(attendance attendance)
    {
        string sqlCommand = "attendanceDelete";
        Sqlconcontrol[] param = {

```

```

new Sqlconcontrol("@att_id", SqlDbType.Int)
};
        param[0].Value = attendance.att_id;
        return
            SqlHelper.ExecuteNonQuery(Conn.SqlConn,
CommandType.StoredProcedure, sqlCommand, param);
    }
    public int Delete(SqlTransaction sp, int att_id)
    {
        string sqlCommand = "attendanceDelete";
        Sqlconcontrol[] param = {
new Sqlconcontrol("@att_id", SqlDbType.Int)
};
        param[0].Value = att_id;
        return SqlHelper.ExecuteNonQuery(sp, CommandType.StoredProcedure,
sqlCommand, param);
    }
    public int Delete(SqlTransaction sp, attendance attendance)
    {
        string sqlCommand = "attendanceDelete";
        Sqlconcontrol[] param = {
new Sqlconcontrol("@att_id", SqlDbType.Int)
};
        param[0].Value = attendance.att_id;
        return SqlHelper.ExecuteNonQuery(sp, CommandType.StoredProcedure,
sqlCommand, param);
    }
    public attendance Select(DataRow row)
    {
        attendance obj = new attendance();
        if (row != null)
        {
            obj.att_id = ((row["att_id"]) == DBNull.Value) ? 0 :
Convert.ToInt32(row["att_id"]);
            obj.att_date = ((row["att_date"]) == DBNull.Value) ?
DateTime.MinValue : Convert.ToDateTime(row["att_date"]);
            obj.att_forenoontime = ((row["att_forenoontime"]) ==
DBNull.Value) ? DateTime.MinValue : Convert.ToDateTime(row["att_forenoontime"]);
            obj.att_afternoontime = ((row["att_afternoontime"]) ==
DBNull.Value) ? DateTime.MinValue : Convert.ToDateTime(row["att_afternoontime"]);
            obj.att_u_id = ((row["att_u_id"]) == DBNull.Value) ? 0 :
Convert.ToInt32(row["att_u_id"]);
            obj.att_dept_id = ((row["att_dept_id"]) == DBNull.Value) ? 0 :
Convert.ToInt32(row["att_dept_id"]);
        }
        else
        {
            return null;
        }
        return obj;
    }
}

```



```

        public attendance Select(IDataReader dr)
        {
            attendance obj = new attendance();
            obj.att_id = ((dr["att_id"] == DBNull.Value) ? 0 :
Convert.ToInt32(dr["att_id"]));
            obj.att_date = ((dr["att_date"] == DBNull.Value) ? DateTime.MinValue :
Convert.ToDateTime(dr["att_date"]));
            obj.att_forenoontime = ((dr["att_forenoontime"] == DBNull.Value) ?
DateTime.MinValue : Convert.ToDateTime(dr["att_forenoontime"]));
            obj.att_afternoontime = ((dr["att_afternoontime"] == DBNull.Value) ?
DateTime.MinValue : Convert.ToDateTime(dr["att_afternoontime"]));
            obj.att_u_id = ((dr["att_u_id"] == DBNull.Value) ? 0 :
Convert.ToInt32(dr["att_u_id"]));
            obj.att_dept_id = ((dr["att_dept_id"] == DBNull.Value) ? 0 :
Convert.ToInt32(dr["att_dept_id"]));
            return obj;
        }

        public attendance Select(int att_id)
        {
            return this.Select(att_id, false, false);
        }

        public attendance Select(int att_id, bool bParentTable, bool
bChildrenTable)
        {
            attendance obj = null;
            Sqlconcontrol[] param = {
new Sqlconcontrol("@att_id", SqlDbType.Int)
};

            param[0].Value = att_id;
            string sqlCommand = "attendanceSelect";
            using (SqlDataReader dr = SqlHelper.ExecuteReader(Conn.SqlConn,
CommandType.StoredProcedure, sqlCommand, param))
            {
                while (dr.Read())
                {
                    obj = this.Select(dr);
                }
            }
            this.Select(obj, bParentTable, bChildrenTable);
            return obj;
        }

        public void Select attendance obj, bool bParentTable, bool bChildrenTable)
        {
            if (bParentTable)
            {
                obj.users = usersBLL.Select(obj.att_u_id);
                obj.department = departmentBLL.Select(obj.att_dept_id);
            }
            if (bChildrenTable)
            {

```

```

    }
}
public void users(attendance attendance)
{
    attendance.users = usersBLL.Select(attendance.att_u_id);
}
public void department(attendance attendance)
{
    attendance.department =
departmentBLL.Select(attendance.att_dept_id);
}
public IList<attendance> Select()
{
    return this.Select(false, false);
}
public IList<attendance> Select(bool bParentTable, bool bChildrenTable)
{
    IList<attendance> list = new List<attendance>();
    string sqlCommand = "attendanceSelectAll";
    using (SqlDataReader dr = SqlHelper.ExecuteReader(Conn.SqlConn,
CommandType.StoredProcedure, sqlCommand))
    {
        while (dr.Read())
        {
            list.Add(this.Select(dr));
        }
    }
    foreach (attendance obj in list)
    {
        this.Select(obj, bParentTable, bChildrenTable);
    }
    return list;
}
public IList<attendance> Select(string where)
{
    return this.Select(where, false, false);
}
public IList<attendance> Select(string where, bool bParentTable, bool
bChildrenTable)
{
    IList<attendance> list = new List<attendance>();
    Sqlconcontrol[] param ={
new Sqlconcontrol("@where", SqlDbType.VarChar, 8000)
};
    param[0].Value = where;
    string sqlCommand = "attendanceSelectByParams";
    using (SqlDataReader dr = SqlHelper.ExecuteReader(Conn.SqlConn,
CommandType.StoredProcedure, sqlCommand, param))
    {
        while (dr.Read())

```

```

        {
            list.Add(this.Select(dr));
        }
    }
    foreach (attendance obj in list)
    {
        this.Select(obj, bParentTable, bChildrenTable);
    }
    return list;
}

public IList<attendance> Select(string foreignFieldName, int
foreignFieldValue)
{
    return this.Select(foreignFieldName, foreignFieldValue, false,
false);
}

public IList<attendance> Select(string foreignFieldName, int
foreignFieldValue, bool bParentTable, bool bChildrenTable)
{
    return this.Select(string.Format("{0}='{1}'", foreignFieldName,
foreignFieldValue), bParentTable, bChildrenTable);
}

public void Select(string where, out int recordCount)
{
    string sqlCommand = "attendanceCountByWhere";
    Sqlconcontrol[] param = {
new Sqlconcontrol("@where", SqlDbType.VarChar, 8000),
new Sqlconcontrol("@recordCount", SqlDbType.Int)
};

    param[0].Value = where;
    param[1].Direction = concontrolDirection.Output;
    SqlHelper.ExecuteNonQuery(Conn.SqlConn, CommandType.StoredProcedure,
sqlCommand, param);
    recordCount = Convert.ToInt32(param[1].Value);
}

public IList<attendance> Select(int pageSize, int pageIndex, string where)
{
    return this.Select(pageSize, pageIndex, where, false, false);
}

public IList<attendance> Select(int pageSize, int pageIndex, string where,
bool bParentTable, bool bChildrenTable)
{
    IList<attendance> list = new List<attendance>();
    string sqlCommand = "attendanceSelectByPagerParams";
    Sqlconcontrol[] param = {
new Sqlconcontrol("@pageSize", SqlDbType.Int),
new Sqlconcontrol("@pageIndex", SqlDbType.Int),
new Sqlconcontrol("@where", SqlDbType.VarChar, 8000)
};

    param[0].Value = pageSize;

```

```

        param[1].Value = pageIndex;
        param[2].Value = where;
        using (SqlDataReader dr = SqlHelper.ExecuteReader(Conn.SqlConn,
CommandType.StoredProcedure, sqlCommand, param))
        {
            while (dr.Read())
            {
                list.Add(this.Select(dr));
            }
        }
        foreach (attendance obj in list)
        {
            this.Select(obj, bParentTable, bChildrenTable);
        }
        return list;
    }

    public IList<attendance> Select(CommandType commandType, string
sqlCommand, params Sqlconcontrol[] param)
    {
        return this.Select(false, false, commandType, sqlCommand, param);
    }

    public IList<attendance> Select(bool bParentTable, bool bChildrenTable,
CommandType commandType, string sqlCommand, params Sqlconcontrol[] param)
    {
        IList<attendance> list = new List<attendance>();
        using (SqlDataReader dr = SqlHelper.ExecuteReader(Conn.SqlConn,
commandType, sqlCommand, param))
        {
            while (dr.Read())
            {
                list.Add(this.Select(dr));
            }
        }
        foreach (attendance obj in list)
        {
            this.Select(obj, bParentTable, bChildrenTable);
        }
        return list;
    }

    public bool Exists(int att_id)
    {
        Sqlconcontrol[] param = {
            new Sqlconcontrol("@att_id", SqlDbType.Int)
        };
        param[0].Value = att_id;
        string sqlCommand = "attendanceIsExist";
        int a = Convert.ToInt32(SqlHelper.ExecuteScalar(Conn.SqlConn,
CommandType.StoredProcedure, sqlCommand, param));
        if (a > 0)
        {

```

```

        return true;
    }
    else
    {
        return false;
    }
}
}
}
using System;
using System.Collections;
using System.controller.data;
using System.Data;
using System.Data.SqlClient;
using System.Text;
using controller.system;
using controller.system.I;
using controller.system.BLL;
namespace controller.system.SqlServer
{
    public partial class bbs : Ibbs
    {
        public int Insert(bbs bbs)
        {
            string sqlCommand = "bbsInsert";
            int res;
            Sqlconcontrol[] param = {
new Sqlconcontrol("@bbs_id", SqlDbType.Int),
new Sqlconcontrol("@bbs_title", SqlDbType.VarChar),
new Sqlconcontrol("@bbs_content", SqlDbType.VarChar),
new Sqlconcontrol("@bbs_photo", SqlDbType.VarChar),
new Sqlconcontrol("@bbs_author", SqlDbType.VarChar),
new Sqlconcontrol("@bbs_releasetime", SqlDbType.DateTime),
new Sqlconcontrol("@bbs_dept_id", SqlDbType.Int),
new Sqlconcontrol("@bbs_reverts", SqlDbType.Int),
new Sqlconcontrol("@bbs_reads", SqlDbType.Int)
};

            param[0].Direction = concontrolDirection.Output;
            param[1].Value = bbs.bbs_title;
            param[2].Value = bbs.bbs_content;
            param[3].Value = bbs.bbs_photo;
            param[4].Value = bbs.bbs_author;
            param[5].Value = bbs.bbs_releasetime;
            param[6].Value = bbs.bbs_dept_id;
            param[7].Value = bbs.bbs_reverts;
            param[8].Value = bbs.bbs_reads;
            res = SqlHelper.ExecuteNonQuery(Conn.SqlConn,
CommandType.StoredProcedure, sqlCommand, param);
            bbs.bbs_id = ((param[0].Value == DBNull.Value) ? 0 :
Convert.ToInt32(param[0].Value));

```

```

        return res;
    }

    public int Insert(SqlTransaction sp, bbs bbs)
    {
        string sqlCommand = "bbsInsert";
        int res;
        Sqlconcontrol[] param = {
new Sqlconcontrol("@bbs_id", SqlDbType.Int),
new Sqlconcontrol("@bbs_title", SqlDbType.VarChar),
new Sqlconcontrol("@bbs_content", SqlDbType.VarChar),
new Sqlconcontrol("@bbs_photo", SqlDbType.VarChar),
new Sqlconcontrol("@bbs_author", SqlDbType.VarChar),
new Sqlconcontrol("@bbs_releasetime", SqlDbType.DateTime),
new Sqlconcontrol("@bbs_dept_id", SqlDbType.Int),
new Sqlconcontrol("@bbs_reverts", SqlDbType.Int),
new Sqlconcontrol("@bbs_reads", SqlDbType.Int)
};

        param[0].Direction = concontrolDirection.Output;
        param[1].Value = bbs.bbs_title;
        param[2].Value = bbs.bbs_content;
        param[3].Value = bbs.bbs_photo;
        param[4].Value = bbs.bbs_author;
        param[5].Value = bbs.bbs_releasetime;
        param[6].Value = bbs.bbs_dept_id;
        param[7].Value = bbs.bbs_reverts;
        param[8].Value = bbs.bbs_reads;
        res = SqlHelper.ExecuteNonQuery(sp, CommandType.StoredProcedure,
sqlCommand, param);
        bbs.bbs_id = ((param[0].Value == DBNull.Value) ? 0 :
Convert.ToInt32(param[0].Value));
        return res;
    }

    public int Update(bbs bbs)
    {
        string sqlCommand = "bbsUpdate";
        Sqlconcontrol[] param = {
new Sqlconcontrol("@bbs_id", SqlDbType.Int),
new Sqlconcontrol("@bbs_title", SqlDbType.VarChar),
new Sqlconcontrol("@bbs_content", SqlDbType.VarChar),
new Sqlconcontrol("@bbs_photo", SqlDbType.VarChar),
new Sqlconcontrol("@bbs_author", SqlDbType.VarChar),
new Sqlconcontrol("@bbs_releasetime", SqlDbType.DateTime),
new Sqlconcontrol("@bbs_dept_id", SqlDbType.Int),
new Sqlconcontrol("@bbs_reverts", SqlDbType.Int),
new Sqlconcontrol("@bbs_reads", SqlDbType.Int)
};

        param[0].Value = bbs.bbs_id;
        param[1].Value = bbs.bbs_title;
        param[2].Value = bbs.bbs_content;
        param[3].Value = bbs.bbs_photo;

```

```

        param[4].Value = bbs.bbs_author;
        param[5].Value = bbs.bbs_releasetime;
        param[6].Value = bbs.bbs_dept_id;
        param[7].Value = bbs.bbs_reverts;
        param[8].Value = bbs.bbs_reads;
        return SqlHelper.ExecuteNonQuery(Conn.SqlConn,
CommandType.StoredProcedure, sqlCommand, param);
    }

    public int Update(SqlTransaction sp, bbs bbs)
    {
        string sqlCommand = "bbsUpdate";
        Sqlconcontrol[] param = {
new Sqlconcontrol("@bbs_id", SqlDbType.Int),
new Sqlconcontrol("@bbs_title", SqlDbType.VarChar),
new Sqlconcontrol("@bbs_content", SqlDbType.VarChar),
new Sqlconcontrol("@bbs_photo", SqlDbType.VarChar),
new Sqlconcontrol("@bbs_author", SqlDbType.VarChar),
new Sqlconcontrol("@bbs_releasetime", SqlDbType.DateTime),
new Sqlconcontrol("@bbs_dept_id", SqlDbType.Int),
new Sqlconcontrol("@bbs_reverts", SqlDbType.Int),
new Sqlconcontrol("@bbs_reads", SqlDbType.Int)
};

        param[0].Value = bbs.bbs_id;
        param[1].Value = bbs.bbs_title;
        param[2].Value = bbs.bbs_content;
        param[3].Value = bbs.bbs_photo;
        param[4].Value = bbs.bbs_author;
        param[5].Value = bbs.bbs_releasetime;
        param[6].Value = bbs.bbs_dept_id;
        param[7].Value = bbs.bbs_reverts;
        param[8].Value = bbs.bbs_reads;
        return SqlHelper.ExecuteNonQuery(sp, CommandType.StoredProcedure,
sqlCommand, param);
    }

    public int Delete(int bbs_id)
    {
        string sqlCommand = "bbsDelete";
        Sqlconcontrol[] param = {
new Sqlconcontrol("@bbs_id", SqlDbType.Int)
};

        param[0].Value = bbs_id;
        return SqlHelper.ExecuteNonQuery(Conn.SqlConn,
CommandType.StoredProcedure, sqlCommand, param);
    }

    public int Delete(bbs bbs)
    {
        string sqlCommand = "bbsDelete";
        Sqlconcontrol[] param = {
new Sqlconcontrol("@bbs_id", SqlDbType.Int)
};

```

```

        param[0].Value = bbs.bbs_id;
        return SqlHelper.ExecuteNonQuery(Conn.SqlConn,
CommandType.StoredProcedure, sqlCommand, param);
    }
    public int Delete(SqlTransaction sp, int bbs_id)
    {
        string sqlCommand = "bbsDelete";
        SqlParameter[] param = {
new SqlParameter("@bbs_id", SqlDbType.Int)
};
        param[0].Value = bbs_id;
        return SqlHelper.ExecuteNonQuery(sp, CommandType.StoredProcedure,
sqlCommand, param);
    }
    public int Delete(SqlTransaction sp, bbs bbs)
    {
        string sqlCommand = "bbsDelete";
        SqlParameter[] param = {
new SqlParameter("@bbs_id", SqlDbType.Int)
};
        param[0].Value = bbs.bbs_id;
        return SqlHelper.ExecuteNonQuery(sp, CommandType.StoredProcedure,
sqlCommand, param);
    }
    public bbs Select(DataRow row)
    {
        bbs obj = new bbs();
        if (row != null)
        {
            obj.bbs_id = ((row["bbs_id"] == DBNull.Value) ? 0 :
Convert.ToInt32(row["bbs_id"]));
            obj.bbs_title = row["bbs_title"].ToString();
            obj.bbs_content = row["bbs_content"].ToString();
            obj.bbs_photo = row["bbs_photo"].ToString();
            obj.bbs_author = row["bbs_author"].ToString();
            obj.bbs_releasetime = ((row["bbs_releasetime"] == DBNull.Value) ?
DateTime.MinValue : Convert.ToDateTime(row["bbs_releasetime"]));
            obj.bbs_dept_id = ((row["bbs_dept_id"] == DBNull.Value) ? 0 :
Convert.ToInt32(row["bbs_dept_id"]));
            obj.bbs_reverts = ((row["bbs_reverts"] == DBNull.Value) ? 0 :
Convert.ToInt32(row["bbs_reverts"]));
            obj.bbs_reads = ((row["bbs_reads"] == DBNull.Value) ? 0 :
Convert.ToInt32(row["bbs_reads"]));
        }
        else
        {
            return null;
        }
        return obj;
    }
}

```



```

        public bbs Select(IDataReader dr)
        {
            bbs obj = new bbs();
            obj.bbs_id = ((dr["bbs_id"] == DBNull.Value) ? 0 :
Convert.ToInt32(dr["bbs_id"]));
            obj.bbs_title = dr["bbs_title"].ToString();
            obj.bbs_content = dr["bbs_content"].ToString();
            obj.bbs_photo = dr["bbs_photo"].ToString();
            obj.bbs_author = dr["bbs_author"].ToString();
            obj.bbs_releasetime = ((dr["bbs_releasetime"] == DBNull.Value) ?
DateTime.MinValue : Convert.ToDateTime(dr["bbs_releasetime"]));
            obj.bbs_dept_id = ((dr["bbs_dept_id"] == DBNull.Value) ? 0 :
Convert.ToInt32(dr["bbs_dept_id"]));
            obj.bbs_reverts = ((dr["bbs_reverts"] == DBNull.Value) ? 0 :
Convert.ToInt32(dr["bbs_reverts"]));
            obj.bbs_reads = ((dr["bbs_reads"] == DBNull.Value) ? 0 :
Convert.ToInt32(dr["bbs_reads"]));
            return obj;
        }

        public bbs Select(int bbs_id)
        {
            return this.Select(bbs_id, false, false);
        }

        public bbs Select(int bbs_id, bool bParentTable, bool bChildrenTable)
        {
            bbs obj = null;
            Sqlconcontrol[] param = {
new Sqlconcontrol("@bbs_id", SqlDbType.Int)
};

            param[0].Value = bbs_id;
            string sqlCommand = "bbsSelect";
            using (SqlDataReader dr = SqlHelper.ExecuteReader(Conn.SqlConn,
CommandType.StoredProcedure, sqlCommand, param))
            {
                while (dr.Read())
                {
                    obj = this.Select(dr);
                }
            }
            this.Select(obj, bParentTable, bChildrenTable);
            return obj;
        }

        public void Select(bbs obj, bool bParentTable, bool bChildrenTable)
        {
            if (bParentTable)
            {
                obj.department = departmentBLL.Select(obj.bbs_dept_id);
            }
            if (bChildrenTable)
            {

```

```
        obj.bbsrevert = bbsrevertBLL.Select("br_bbs_id=" +
obj.bbs_id.ToString());
        foreach (bbsrevert item in obj.bbsrevert)
        {
            item.bbs = obj;
        }
    }
}
public void department(bbs bbs)
{
    bbs.department = departmentBLL.Select(bbs.bbs_dept_id);
}
public void bbsrevert(bbs bbs)
{
    bbs.bbsrevert = bbsrevertBLL.Select("br_bbs_id=" +
bbs.bbs_id.ToString());
}
public IList<bbs> Select()
{
    return this.Select(false, false);
}
public IList<bbs> Select(bool bParentTable, bool bChildrenTable)
{
    IList<bbs> list = new List<bbs>();
    string sqlCommand = "bbsSelectAll";
    using (SqlDataReader dr = SqlHelper.ExecuteReader(Conn.SqlConn,
CommandType.StoredProcedure, sqlCommand))
    {
        while (dr.Read())
        {
            list.Add(this.Select(dr));
        }
    }
    foreach (bbs obj in list)
    {
        this.Select(obj, bParentTable, bChildrenTable);
    }
    return list;
}
public IList<bbs> Select(string where)
{
    return this.Select(where, false, false);
}
public IList<bbs> Select(string where, bool bParentTable, bool
bChildrenTable)
{
    IList<bbs> list = new List<bbs>();
    Sqlconcontrol[] param = {
new Sqlconcontrol("@where", SqlDbType.VarChar, 8000)
};
};
```

```

        param[0].Value = where;
        string sqlCommand = "bbsSelectByParams";
        using (SqlDataReader dr = SqlHelper.ExecuteReader(Conn.SqlConn,
CommandType.StoredProcedure, sqlCommand, param))
        {
            while (dr.Read())
            {
                list.Add(this.Select(dr));
            }
        }
        foreach (bbs obj in list)
        {
            this.Select(obj, bParentTable, bChildrenTable);
        }
        return list;
    }

    public IList<bbs> Select(string foreignFieldName, int foreignFieldValue)
    {
        return this.Select(foreignFieldName, foreignFieldValue, false,
false);
    }

    public IList<bbs> Select(string foreignFieldName, int foreignFieldValue,
bool bParentTable, bool bChildrenTable)
    {
        return this.Select(string.Format("{0}='{1}'", foreignFieldName,
foreignFieldValue), bParentTable, bChildrenTable);
    }

    public void Select(string where, out int recordCount)
    {
        string sqlCommand = "bbsCountByWhere";
        Sqlconcontrol[] param = {
new Sqlconcontrol("@where", SqlDbType.VarChar, 8000),
new Sqlconcontrol("@recordCount", SqlDbType.Int)
};

        param[0].Value = where;
        param[1].Direction = concontrolDirection.Output;
        SqlHelper.ExecuteNonQuery(Conn.SqlConn, CommandType.StoredProcedure,
sqlCommand, param);
        recordCount = Convert.ToInt32(param[1].Value);
    }

    public IList<bbs> Select(int pageSize, int pageIndex, string where)
    {
        return this.Select(pageSize, pageIndex, where, false, false);
    }

    public IList<bbs> Select(int pageSize, int pageIndex, string where, bool
bParentTable, bool bChildrenTable)
    {
        IList<bbs> list = new List<bbs>();
        string sqlCommand = "bbsSelectByPagerParams";
        Sqlconcontrol[] param = {

```

```
new Sqlconcontrol("@pageSize", SqlDbType.Int),
new Sqlconcontrol("@pageIndex", SqlDbType.Int),
new Sqlconcontrol("@where", SqlDbType.VarChar, 8000)
};

        param[0].Value = pageSize;
        param[1].Value = pageIndex;
        param[2].Value = where;
        using (SqlDataReader dr = SqlHelper.ExecuteReader(Conn.SqlConn,
CommandType.StoredProcedure, sqlCommand, param))
        {
            while (dr.Read())
            {
                list.Add(this.Select(dr));
            }
        }
        foreach (bbs obj in list)
        {
            this.Select(obj, bParentTable, bChildrenTable);
        }
        return list;
    }

    public IList<bbs> Select(CommandType commandType, string sqlCommand,
params Sqlconcontrol[] param)
    {
        return this.Select(false, false, commandType, sqlCommand, param);
    }

    public IList<bbs> Select(bool bParentTable, bool bChildrenTable,
CommandType commandType, string sqlCommand, params Sqlconcontrol[] param)
    {
        IList<bbs> list = new List<bbs>();
        using (SqlDataReader dr = SqlHelper.ExecuteReader(Conn.SqlConn,
commandType, sqlCommand, param))
        {
            while (dr.Read())
            {
                list.Add(this.Select(dr));
            }
        }
        foreach (bbs obj in list)
        {
            this.Select(obj, bParentTable, bChildrenTable);
        }
        return list;
    }

    public bool Exists(int bbs_id)
    {
        Sqlconcontrol[] param = {
            new Sqlconcontrol("@bbs_id", SqlDbType.Int)
        };
        param[0].Value = bbs_id;
```

```

        string sqlCommand = "bbsIsExist";
        int a = Convert.ToInt32(SqlHelper.ExecuteScalar(Conn.SqlConn,
CommandType.StoredProcedure, sqlCommand, param));
        if (a > 0)
        {
            return true;
        }
        else
        {
            return false;
        }
    }
}

using System;
using System.Collections;
using System.controller.data;
using System.Data;
using System.Data.SqlClient;
using System.Text;
using controller.system;
using controller.system.I;
using controller.system.BLL;
namespace controller.system.SqlServer
{
    public bbsrevert()
    {
    }

    public int Insert(bbsrevert bbsrevert)
    {
        string sqlCommand = "bbsrevertInsert";
        int res;
        Sqlconcontrol[] param = {
new Sqlconcontrol("@br_id", SqlDbType.Int),
new Sqlconcontrol("@br_bbs_id", SqlDbType.Int),
new Sqlconcontrol("@br_revertcontent", SqlDbType.VarChar),
new Sqlconcontrol("@br_reverttime", SqlDbType.DateTime),
new Sqlconcontrol("@br_u_id", SqlDbType.Int)
};

        param[0].Direction = concontrolDirection.Output;
        param[1].Value = bbsrevert.br_bbs_id;
        param[2].Value = bbsrevert.br_revertcontent;
        param[3].Value = bbsrevert.br_reverttime;
        param[4].Value = bbsrevert.br_u_id;
        res = SqlHelper.ExecuteNonQuery(Conn.SqlConn,
CommandType.StoredProcedure, sqlCommand, param);
        bbsrevert.br_id = ((param[0].Value) == DBNull.Value) ? 0 :
Convert.ToInt32(param[0].Value);
        return res;
    }
}

```

```

        public int Insert(SqlTransaction sp, bbsrevert bbsrevert)
        {
            string sqlCommand = "bbsrevertInsert";
            int res;
            Sqlconcontrol[] param = {
new Sqlconcontrol("@br_id", SqlDbType.Int),
new Sqlconcontrol("@br_bbs_id", SqlDbType.Int),
new Sqlconcontrol("@br_revertcontent", SqlDbType.VarChar),
new Sqlconcontrol("@br_reverttime", SqlDbType.DateTime),
new Sqlconcontrol("@br_u_id", SqlDbType.Int)
};

            param[0].Direction = concontrolDirection.Output;
            param[1].Value = bbsrevert.br_bbs_id;
            param[2].Value = bbsrevert.br_revertcontent;
            param[3].Value = bbsrevert.br_reverttime;
            param[4].Value = bbsrevert.br_u_id;
            res = SqlHelper.ExecuteNonQuery(sp, CommandType.StoredProcedure,
sqlCommand, param);
            bbsrevert.br_id = ((param[0].Value) == DBNull.Value) ? 0 :
Convert.ToInt32(param[0].Value);
            return res;
        }

        public int Update(bbsrevert bbsrevert)
        {
            string sqlCommand = "bbsrevertUpdate";
            Sqlconcontrol[] param = {
new Sqlconcontrol("@br_id", SqlDbType.Int),
new Sqlconcontrol("@br_bbs_id", SqlDbType.Int),
new Sqlconcontrol("@br_revertcontent", SqlDbType.VarChar),
new Sqlconcontrol("@br_reverttime", SqlDbType.DateTime),
new Sqlconcontrol("@br_u_id", SqlDbType.Int)
};

            return SqlHelper.ExecuteNonQuery(Conn.SqlConn,
CommandType.StoredProcedure, sqlCommand, param);
        }

        public int Update(SqlTransaction sp, bbsrevert bbsrevert)
        {
            string sqlCommand = "bbsrevertUpdate";
            Sqlconcontrol[] param = {
new Sqlconcontrol("@br_id", SqlDbType.Int),
new Sqlconcontrol("@br_bbs_id", SqlDbType.Int),
new Sqlconcontrol("@br_revertcontent", SqlDbType.VarChar),
new Sqlconcontrol("@br_reverttime", SqlDbType.DateTime),
new Sqlconcontrol("@br_u_id", SqlDbType.Int)
};

            param[0].Value = bbsrevert.br_id;
            param[1].Value = bbsrevert.br_bbs_id;
            param[2].Value = bbsrevert.br_revertcontent;
            param[3].Value = bbsrevert.br_reverttime;
            param[4].Value = bbsrevert.br_u_id;

```

```

        return SqlHelper.ExecuteNonQuery(sp, CommandType.StoredProcedure,
sqlCommand, param);
    }
    public int Delete(int br_id)
    {
        string sqlCommand = "bbsrevertDelete";
        Sqlconcontrol[] param = {
new Sqlconcontrol("@br_id", SqlDbType.Int)
};
        param[0].Value = br_id;
        return SqlHelper.ExecuteNonQuery(Conn. SqlConnection,
CommandType.StoredProcedure, sqlCommand, param);
    }
    public int Delete(bbsrevert bbsrevert)
    {
        string sqlCommand = "bbsrevertDelete";
        Sqlconcontrol[] param = {
new Sqlconcontrol("@br_id", SqlDbType.Int)
};
        param[0].Value = bbsrevert.br_id;
        return SqlHelper.ExecuteNonQuery(Conn. SqlConnection,
CommandType.StoredProcedure, sqlCommand, param);
    }
    public int Delete(SqlTransaction sp, int br_id)
    {
        string sqlCommand = "bbsrevertDelete";
        Sqlconcontrol[] param = {
new Sqlconcontrol("@br_id", SqlDbType.Int)
};
        param[0].Value = br_id;
        return SqlHelper.ExecuteNonQuery(sp, CommandType.StoredProcedure,
sqlCommand, param);
    }
    public int Delete(SqlTransaction sp, bbsrevert bbsrevert)
    {
        string sqlCommand = "bbsrevertDelete";
        Sqlconcontrol[] param = {
new Sqlconcontrol("@br_id", SqlDbType.Int)
};
        param[0].Value = bbsrevert.br_id;
        return SqlHelper.ExecuteNonQuery(sp, CommandType.StoredProcedure,
sqlCommand, param);
    }
    public bbsrevert Select(DataRow row)
    {
        bbsrevert obj = new bbsrevert();
        if (row != null)
        {
            obj.br_id = ((row["br_id"]) == DBNull.Value) ? 0 :
Convert.ToInt32(row["br_id"]);

```

```

        obj.br_bbs_id = ((row["br_bbs_id"]) == DBNull.Value) ? 0 :
Convert.ToInt32(row["br_bbs_id"]);
        obj.br_revertcontent = row["br_revertcontent"].ToString();
        obj.br_reverttime = ((row["br_reverttime"]) == DBNull.Value) ?
DateTime.MinValue : Convert.ToDateTime(row["br_reverttime"]);
        obj.br_u_id = ((row["br_u_id"]) == DBNull.Value) ? 0 :
Convert.ToInt32(row["br_u_id"]);
    }
    else
    {
        return null;
    }
    return obj;
}

public bbsrevert Select(IDataReader dr)
{
    bbsrevert obj = new bbsrevert();
    obj.br_id = ((dr["br_id"]) == DBNull.Value) ? 0 :
Convert.ToInt32(dr["br_id"]);
    obj.br_bbs_id = ((dr["br_bbs_id"]) == DBNull.Value) ? 0 :
Convert.ToInt32(dr["br_bbs_id"]);
    obj.br_revertcontent = dr["br_revertcontent"].ToString();
    obj.br_reverttime = ((dr["br_reverttime"]) == DBNull.Value) ?
DateTime.MinValue : Convert.ToDateTime(dr["br_reverttime"]);
    obj.br_u_id = ((dr["br_u_id"]) == DBNull.Value) ? 0 :
Convert.ToInt32(dr["br_u_id"]);
    return obj;
}

public bbsrevert Select(int br_id)
{
    return this.Select(br_id, false, false);
}

public bbsrevert Select(int br_id, bool bParentTable, bool bChildrenTable)
{
    bbsrevert obj = null;
    Sqlconcontrol[] param = {
new Sqlconcontrol("@br_id", SqlDbType.Int)
};
    param[0].Value = br_id;
    string sqlCommand = "bbsrevertSelect";
    using (SqlDataReader dr = SqlHelper.ExecuteReader(Conn.SqlConn,
CommandType.StoredProcedure, sqlCommand, param))
    {
        while (dr.Read())
        {
            obj = this.Select(dr);
        }
    }
    this.Select(obj, bParentTable, bChildrenTable);
    return obj;
}

```



```
}
public void Select(bbsrevert obj, bool bParentTable, bool bChildrenTable)
{
    if (bParentTable)
    {
        obj.users = usersBLL.Select(obj.br_u_id);
        obj.bbs = bbsBLL.Select(obj.br_bbs_id);
    }
    if (bChildrenTable)
    {
    }
}
public void users(bbsrevert bbsrevert)
{
    bbsrevert.users = usersBLL.Select(bbsrevert.br_u_id);
}
public void bbs(bbsrevert bbsrevert)
{
    bbsrevert.bbs = bbsBLL.Select(bbsrevert.br_bbs_id);
}
public IList<bbsrevert> Select()
{
    return this.Select(false, false);
}
public IList<bbsrevert> Select(bool bParentTable, bool bChildrenTable)
{
    IList<bbsrevert> list = new List<bbsrevert>();
    string sqlCommand = "bbsrevertSelectAll";
    using (SqlDataReader dr = SqlHelper.ExecuteReader(Conn.SqlConn,
CommandType.StoredProcedure, sqlCommand))
    {
        while (dr.Read())
        {
            list.Add(this.Select(dr));
        }
    }
    foreach (bbsrevert obj in list)
    {
        this.Select(obj, bParentTable, bChildrenTable);
    }
    return list;
}
public IList<bbsrevert> Select(string where)
{
    return this.Select(where, false, false);
}
public IList<bbsrevert> Select(string where, bool bParentTable, bool
bChildrenTable)
{
    IList<bbsrevert> list = new List<bbsrevert>();
```

```

        Sqlconcontrol[] param = {
new Sqlconcontrol("@where", SqlDbType.VarChar, 8000)
};

        param[0].Value = where;
        string sqlCommand = "bbsrevertSelectByParams";
        using (SqlDataReader dr = SqlHelper.ExecuteReader(Conn.SqlConn,
CommandType.StoredProcedure, sqlCommand, param))
        {
            while (dr.Read())
            {
                list.Add(this.Select(dr));
            }
        }
        foreach (bbsrevert obj in list)
        {
            this.Select(obj, bParentTable, bChildrenTable);
        }
        return list;
    }

    public IList<bbsrevert> Select(string foreignFieldName, int
foreignFieldValue)
    {
        return this.Select(foreignFieldName, foreignFieldValue, false,
false);
    }

    public IList<bbsrevert> Select(string foreignFieldName, int
foreignFieldValue, bool bParentTable, bool bChildrenTable)
    {
        return this.Select(string.Format("{0}='{1}'", foreignFieldName,
foreignFieldValue), bParentTable, bChildrenTable);
    }

    public void Select(string where, out int recordCount)
    {
        string sqlCommand = "bbsrevertCountByWhere";
        Sqlconcontrol[] param = {
new Sqlconcontrol("@where", SqlDbType.VarChar, 8000),
new Sqlconcontrol("@recordCount", SqlDbType.Int)
};

        param[0].Value = where;
        param[1].Direction = concontrolDirection.Output;
        SqlHelper.ExecuteNonQuery(Conn.SqlConn, CommandType.StoredProcedure,
sqlCommand, param);
        recordCount = Convert.ToInt32(param[1].Value);
    }

    public IList<bbsrevert> Select(int pageSize, int pageIndex, string where)
    {
        return this.Select(pageSize, pageIndex, where, false, false);
    }

    public IList<bbsrevert> Select(int pageSize, int pageIndex, string where,
bool bParentTable, bool bChildrenTable)

```

```

    {
        IList<bbsrevert> list = new List<bbsrevert>();
        string sqlCommand = "bbsrevertSelectByPagerParams";
        Sqlconcontrol[] param = {
new Sqlconcontrol("@pageSize", SqlDbType.Int),
new Sqlconcontrol("@pageIndex", SqlDbType.Int),
new Sqlconcontrol("@where", SqlDbType.VarChar, 8000)
};

        param[0].Value = pageSize;
        param[1].Value = pageIndex;
        param[2].Value = where;
        using (SqlDataReader dr = SqlHelper.ExecuteReader(Conn.SqlConn,
CommandType.StoredProcedure, sqlCommand, param))
        {
            while (dr.Read())
            {
                list.Add(this.Select(dr));
            }
        }
        foreach (bbsrevert obj in list)
        {
            this.Select(obj, bParentTable, bChildrenTable);
        }
        return list;
    }

    public IList<bbsrevert> Select(CommandType commandType, string sqlCommand,
params Sqlconcontrol[] param)
    {
        return this.Select(false, false, commandType, sqlCommand, param);
    }

    public IList<bbsrevert> Select(bool bParentTable, bool bChildrenTable,
CommandType commandType, string sqlCommand, params Sqlconcontrol[] param)
    {
        IList<bbsrevert> list = new List<bbsrevert>();
        using (SqlDataReader dr = SqlHelper.ExecuteReader(Conn.SqlConn,
commandType, sqlCommand, param))
        {
            while (dr.Read())
            {
                list.Add(this.Select(dr));
            }
        }
        foreach (bbsrevert obj in list)
        {
            this.Select(obj, bParentTable, bChildrenTable);
        }
        return list;
    }

    public bool Exists(int br_id)
    {

```

```

        Sqlconcontrol[] param = {
            new Sqlconcontrol("@br_id", SqlDbType.Int)
        };
        param[0].Value = br_id;
        string sqlCommand = "bbsrevertIsExist";
        int a = Convert.ToInt32(SqlHelper.ExecuteScalar(Conn.SqlConn,
        CommandType.StoredProcedure, sqlCommand, param));
        if (a > 0)
        {
            return true;
        }
        else
        {
            return false;
        }
    }
}
}
namespace controller.system.SqlServer
{
    {
        public department()
        {
        }
        public int Insert(department department)
        {
            string sqlCommand = "departmentInsert";
            int res;
            Sqlconcontrol[] param = {
new Sqlconcontrol("@dept_id", SqlDbType.Int),
new Sqlconcontrol("@dept_department", SqlDbType.VarChar)
};
            param[0].Direction = concontrolDirection.Output;
            param[1].Value = department.dept_department;
            res = SqlHelper.ExecuteNonQuery(Conn.SqlConn,
        CommandType.StoredProcedure, sqlCommand, param);
            department.dept_id = ((param[0].Value) == DBNull.Value) ? 0 :
        Convert.ToInt32(param[0].Value);
            return res;
        }
        public int Insert(SqlTransaction sp, department department)
        {
            string sqlCommand = "departmentInsert";
            int res;
            Sqlconcontrol[] param = {
new Sqlconcontrol("@dept_id", SqlDbType.Int),
new Sqlconcontrol("@dept_department", SqlDbType.VarChar)
};
            param[0].Direction = concontrolDirection.Output;
            param[1].Value = department.dept_department;

```

```

        res = SqlHelper.ExecuteNonQuery(sp, CommandType.StoredProcedure,
sqlCommand, param);
        department.dept_id = ((param[0].Value) == DBNull.Value) ? 0 :
Convert.ToInt32(param[0].Value);
        return res;
    }

    public int Update(department department)
    {
        string sqlCommand = "departmentUpdate";
        Sqlconcontrol[] param = {
new Sqlconcontrol("@dept_id", SqlDbType.Int),
new Sqlconcontrol("@dept_department", SqlDbType.VarChar)
};

        param[0].Value = department.dept_id;
        param[1].Value = department.dept_department;
        return
            SqlHelper.ExecuteNonQuery(Conn. SqlConnection,
CommandType.StoredProcedure, sqlCommand, param);
    }

    public int Update(SqlTransaction sp, department department)
    {
        string sqlCommand = "departmentUpdate";
        Sqlconcontrol[] param = {
new Sqlconcontrol("@dept_id", SqlDbType.Int),
new Sqlconcontrol("@dept_department", SqlDbType.VarChar)
};

        param[0].Value = department.dept_id;
        param[1].Value = department.dept_department;
        return SqlHelper.ExecuteNonQuery(sp, CommandType.StoredProcedure,
sqlCommand, param);
    }

    public int Delete(int dept_id)
    {
        string sqlCommand = "departmentDelete";
        Sqlconcontrol[] param = {
new Sqlconcontrol("@dept_id", SqlDbType.Int)
};

        param[0].Value = dept_id;
        return
            SqlHelper.ExecuteNonQuery(Conn. SqlConnection,
CommandType.StoredProcedure, sqlCommand, param);
    }

    public int Delete(department department)
    {
        string sqlCommand = "departmentDelete";
        Sqlconcontrol[] param = {
new Sqlconcontrol("@dept_id", SqlDbType.Int)
};

        param[0].Value = department.dept_id;
        return
            SqlHelper.ExecuteNonQuery(Conn. SqlConnection,
CommandType.StoredProcedure, sqlCommand, param);
    }

```

```

        {
            string sqlCommand = "departmentDelete";
            Sqlconcontrol[] param = {
new Sqlconcontrol("@dept_id", SqlDbType.Int)
};
            param[0].Value = dept_id;
            return SqlHelper.ExecuteNonQuery(sp, CommandType.StoredProcedure,
sqlCommand, param);
        }
        public int Delete(SqlTransaction sp, department department)
        {
            string sqlCommand = "departmentDelete";
            Sqlconcontrol[] param = {
new Sqlconcontrol("@dept_id", SqlDbType.Int)
};
            param[0].Value = department.dept_id;
            return SqlHelper.ExecuteNonQuery(sp, CommandType.StoredProcedure,
sqlCommand, param);
        }
        public department Select(DataRow row)
        {
            department obj = new department();
            if (row != null)
            {
                obj.dept_id = ((row["dept_id"] == DBNull.Value) ? 0 :
Convert.ToInt32(row["dept_id"]));
                obj.dept_department = row["dept_department"].ToString();
            }
            else
            {
                return null;
            }
            return obj;
        }
        public department Select(IDataReader dr)
        {
            department obj = new department();
            obj.dept_id = ((dr["dept_id"] == DBNull.Value) ? 0 :
Convert.ToInt32(dr["dept_id"]));
            obj.dept_department = dr["dept_department"].ToString();
            return obj;
        }
        public department Select(int dept_id)
        {
            return this.Select(dept_id, false, false);
        }
        public department Select(int dept_id, bool bParentTable, bool
bChildrenTable)
        {
            department obj = null;

```

```
        Sqlconcontrol[] param = {
new Sqlconcontrol("@dept_id", SqlDbType.Int)
};

        param[0].Value = dept_id;
        string sqlCommand = "departmentSelect";
        using (SqlDataReader dr = SqlHelper.ExecuteReader(Conn.SqlConn,
CommandType.StoredProcedure, sqlCommand, param))
        {
            while (dr.Read())
            {
                obj = this.Select(dr);
            }
        }
        this.Select(obj, bParentTable, bChildrenTable);
        return obj;
    }

    public void Select(department obj, bool bParentTable, bool bChildrenTable)
    {
        if (bParentTable)
        {
        }
        if (bChildrenTable)
        {
            obj.addresslist = addresslistBLL.Select("al_dept_id=" +
obj.dept_id.ToString());
            foreach (addresslist item in obj.addresslist)
            {
                item.department = obj;
            }
            obj.assetmanage = assetmanageBLL.Select("am_dept_id=" +
obj.dept_id.ToString());
            foreach (assetmanage item in obj.assetmanage)
            {
                item.department = obj;
            }
            obj.attendance = attendanceBLL.Select("att_dept_id=" +
obj.dept_id.ToString());
            foreach (attendance item in obj.attendance)
            {
                item.department = obj;
            }
            obj.bbs = bbsBLL.Select("bbs_dept_id=" + obj.dept_id.ToString());
            foreach (bbs item in obj.bbs)
            {
                item.department = obj;
            }
            obj.document = documentBLL.Select("doc_dept_id=" +
obj.dept_id.ToString());
            foreach (document item in obj.document)
            {

```

```
        item.department = obj;
    }
    obj.control.disease = diseaseBLL.Select("em_dept_id=" +
obj.dept_id.ToString());
    foreach (control.disease item in obj.control.disease)
    {
        item.department = obj;
    }
    obj.examine = examineBLL.Select("ex_dept_id=" +
obj.dept_id.ToString());
    foreach (examine item in obj.examine)
    {
        item.department = obj;
    }
    obj.files = filesBLL.Select("f_dept_id=" +
obj.dept_id.ToString());
    foreach (files item in obj.files)
    {
        item.department = obj;
    }
    obj.news = newsBLL.Select("n_dept_id=" + obj.dept_id.ToString());
    foreach (news item in obj.news)
    {
        item.department = obj;
    }
    obj.sendword = sendwordBLL.Select("sw_dept_id=" +
obj.dept_id.ToString());
    foreach (sendword item in obj.sendword)
    {
        item.department = obj;
    }
    obj.users = usersBLL.Select("u_dept_id=" +
obj.dept_id.ToString());
    foreach (users item in obj.users)
    {
        item.department = obj;
    }
}

public void addresslist(department department)
{
    department.addresslist = addresslistBLL.Select("al_dept_id=" +
department.dept_id.ToString());
}

public void assetmanage(department department)
{
    department.assetmanage = assetmanageBLL.Select("am_dept_id=" +
department.dept_id.ToString());
}

public void attendance(department department)
```



```
{
    department.attendance = attendanceBLL.Select("att_dept_id=" +
department.dept_id.ToString());
}
public void bbs(department department)
{
    department.bbs = bbsBLL.Select("bbs_dept_id=" +
department.dept_id.ToString());
}
public void document(department department)
{
    department.document = documentBLL.Select("doc_dept_id=" +
department.dept_id.ToString());
}
public void disease(department department)
{
    department.control.disease = diseaseBLL.Select("em_dept_id=" +
department.dept_id.ToString());
}
public void examine(department department)
{
    department.examine = examineBLL.Select("ex_dept_id=" +
department.dept_id.ToString());
}
public void files(department department)
{
    department.files = filesBLL.Select("f_dept_id=" +
department.dept_id.ToString());
}
public void news(department department)
{
    department.news = newsBLL.Select("n_dept_id=" +
department.dept_id.ToString());
}
public void sendword(department department)
{
    department.sendword = sendwordBLL.Select("sw_dept_id=" +
department.dept_id.ToString());
}
public void users(department department)
{
    department.users = usersBLL.Select("u_dept_id=" +
department.dept_id.ToString());
}
public IList<department> Select()
{
    return this.Select(false, false);
}
public IList<department> Select(bool bParentTable, bool bChildrenTable)
{

```

```

        IList<department> list = new List<department>>();
        string sqlCommand = "departmentSelectAll";
        using (SqlDataReader dr = SqlHelper.ExecuteReader(Conn.SqlConn,
CommandType.StoredProcedure, sqlCommand))
        {
            while (dr.Read())
            {
                list.Add(this.Select(dr));
            }
        }
        foreach (department obj in list)
        {
            this.Select(obj, bParentTable, bChildrenTable);
        }
        return list;
    }
    public IList<department> Select(string where)
    {
        return this.Select(where, false, false);
    }
    public IList<department> Select(string where, bool bParentTable, bool
bChildrenTable)
    {
        IList<department> list = new List<department>>();
        Sqlconcontrol[] param = {
new Sqlconcontrol("@where", SqlDbType.VarChar, 8000)
};
        param[0].Value = where;
        string sqlCommand = "departmentSelectByParams";
        using (SqlDataReader dr = SqlHelper.ExecuteReader(Conn.SqlConn,
CommandType.StoredProcedure, sqlCommand, param))
        {
            while (dr.Read())
            {
                list.Add(this.Select(dr));
            }
        }
        foreach (department obj in list)
        {
            this.Select(obj, bParentTable, bChildrenTable);
        }
        return list;
    }
    public IList<department> Select(string foreignFieldName, int
foreignFieldValue)
    {
        return this.Select(foreignFieldName, foreignFieldValue, false,
false);
    }
    public IList<department> Select(string foreignFieldName, int

```

```

foreignFieldValue, bool bParentTable, bool bChildrenTable)
{
    return this.Select(string.Format("{0}='{1}'", foreignFieldName,
foreignFieldValue), bParentTable, bChildrenTable);
}

public void Select(string where, out int recordCount)
{
    string sqlCommand = "departmentCountByWhere";
    Sqlconcontrol[] param = {
new Sqlconcontrol("@where", SqlDbType.VarChar, 8000),
new Sqlconcontrol("@recordCount", SqlDbType.Int)
};

    param[0].Value = where;
    param[1].Direction = concontrolDirection.Output;
    SqlHelper.ExecuteNonQuery(Conn.SqlConn, CommandType.StoredProcedure,
sqlCommand, param);
    recordCount = Convert.ToInt32(param[1].Value);
}

public IList<department> Select(int pageSize, int pageIndex, string where)
{
    return this.Select(pageSize, pageIndex, where, false, false);
}

public IList<department> Select(int pageSize, int pageIndex, string where,
bool bParentTable, bool bChildrenTable)
{
    IList<department> list = new List<department>();
    string sqlCommand = "departmentSelectByPagerParams";
    Sqlconcontrol[] param = {
new Sqlconcontrol("@pageSize", SqlDbType.Int),
new Sqlconcontrol("@pageIndex", SqlDbType.Int),
new Sqlconcontrol("@where", SqlDbType.VarChar, 8000)
};

    param[0].Value = pageSize;
    param[1].Value = pageIndex;
    param[2].Value = where;
    using (SqlDataReader dr = SqlHelper.ExecuteReader(Conn.SqlConn,
CommandType.StoredProcedure, sqlCommand, param))
    {
        while (dr.Read())
        {
            list.Add(this.Select(dr));
        }
    }
    foreach (department obj in list)
    {
        this.Select(obj, bParentTable, bChildrenTable);
    }
    return list;
}

public IList<department> Select(CommandType commandType, string

```

```
sqlCommand, params Sqlconcontrol[] param)
{
    return this.Select(false, false, commandType, sqlCommand, param);
}

public IList<department> Select(bool bParentTable, bool bChildrenTable,
CommandType commandType, string sqlCommand, params Sqlconcontrol[] param)
{
    IList<department> list = new List<department>();
    using (SqlDataReader dr = SqlHelper.ExecuteReader(Conn.SqlConn,
commandType, sqlCommand, param))
    {
        while (dr.Read())
        {
            list.Add(this.Select(dr));
        }
    }
    foreach (department obj in list)
    {
        this.Select(obj, bParentTable, bChildrenTable);
    }
    return list;
}

public bool Exists(int dept_id)
{
    Sqlconcontrol[] param = {
        new Sqlconcontrol("@dept_id", SqlDbType.Int)
    };
    param[0].Value = dept_id;
    string sqlCommand = "departmentIsExist";
    int a = Convert.ToInt32(SqlHelper.ExecuteScalar(Conn.SqlConn,
CommandType.StoredProcedure, sqlCommand, param));
    if (a > 0)
    {
        return true;
    }
    else
    {
        return false;
    }
}
}
```